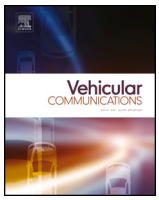
Vehicular Communications 55 (2025) 100956



Contents lists available at ScienceDirect

Vehicular Communications

journal homepage: www.elsevier.com/locate/vehcom





MetaCAN: An optimized adaptive hybrid metaheuristic-based intrusion detection system for CAN bus security

Kadir Ileri [a,b,*], Abdur Rakib [b], Soufiene Djahel [b]

[a] Electrical and Electronics Engineering Department, Bandirma Onyedi Eylul University, Balikesir, 10200, Turkey
[b] Centre for Future Transport and Cities, Coventry University, Coventry, CV1 5FB, UK

ARTICLE INFO

Keywords:
CAN bus
Intrusion detection system
In-vehicle security
Machine learning
Metaheuristic optimization

ABSTRACT

The Controller Area Network (CAN) bus is a message-based protocol widely used in modern vehicles to facilitate communication between various Electronic Control Units (ECUs). However, its simplistic design lacks fundamental security measures, making it highly susceptible to cyberattacks. These vulnerabilities pose significant risks to vehicle safety, highlighting the critical need for implementation of effective intrusion detection systems (IDS). Therefore, in this paper, a machine learning based IDS optimized through an adaptive hybrid metaheuristic approach, named MetaCAN, is proposed to secure the CAN bus. MetaCAN leverages the complementary strengths of particle swarm optimization (PSO) for fast convergence and cuckoo search (CS) for robust global search to ensure effective hyperparameter tuning and model optimization. MetaCAN is evaluated using three real-world datasets including Survival Analysis, Car Hacking: Attack & Defense Challenge 2020, and OTIDS. Unlike traditional binary detection systems, MetaCAN offers multi-class attack detection by identifying five distinct attack types including Denial of Service (DoS), fuzzy, masquerade, malfunction, and replay attacks. Moreover, the detection accuracy of the system is enhanced through a feature engineering process that introduces two effective features such as Time Interval and ID Repetition Count. The experimental results show that MetaCAN consistently outperforms existing IDS solutions targeted the same datasets, making it a promising solution for securing the CAN bus in real-world vehicular environments.

1. Introduction

Automobiles have evolved to meet the demands of modern society by replacing traditional mechanical components with interconnected electronic systems, such as electronic control units (ECU) [1]. This transformation has turned vehicles into intelligent systems, integrating remote communication technologies, adaptive sensors, and data-driven applications to enhance comfort and connectivity. Each ECU is responsible for specific operations, including braking, airbag control, gear shifting, door locking, and stability control. These ECUs are interconnected through a Controller Area Network (CAN), a widely used vehicle network known for its real-time performance, reliability, ease of maintenance, and cost effectiveness [2]. However, while CAN offers these benefits, it also has significant drawbacks, such as lack of authentication, weak access control, and no encryption. Lokman et al. highlight the critical need for cybersecurity in CAN systems, particularly due to the absence of an authentication mechanism [3]. Moreover, the CAN bus's broadcast nature can exacerbate security risks, as any malicious ECU on the network can transmit messages. Together, these factors make the CAN bus susceptible to cyberattacks.

The first known CAN bus attack was performed by Hoppe and Dittman [4] in a simulation environment. It was performed in 2007 by targeting an electric window lift. In 2010, Koscher et al. [5] exploited the CAN bus via the On-Board Diagnostics II (OBD-II) port by gaining control over critical modules such as engine and brake control. In addition, the attack involves the engine being disabled, changing its parameters, and manipulating the instrument cluster to display false data. In the following year, Hoppe et al. [6] performed an attack on the gateway ECU using a black box. It has also been shown that an attacker can eavesdrop on communication on the CAN bus. While the attack did not compromise the CAN bus fully, it could still divert the driver's attention away by displaying a warning message on the screen and triggering an alarm. In 2011, Checkoway et al. [7] demonstrated that a vehicle's radio can be compromised and custom firmware can be uploaded via a

* Corresponding author at: Centre for Future Transport and Cities, Coventry University, Coventry, CV1 5FB, UK.
E-mail addresses: kileri@bandirma.edu.tr (K. Ileri), ad9812@coventry.ac.uk (A. Rakib), ae3095@coventry.ac.uk (S. Djahel).

https://doi.org/10.1016/j.vehcom.2025.100956
Received 25 February 2025; Received in revised form 20 June 2025; Accepted 2 July 2025
Available online 10 July 2025
2214-2096/Crown Copyright © 2025 Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

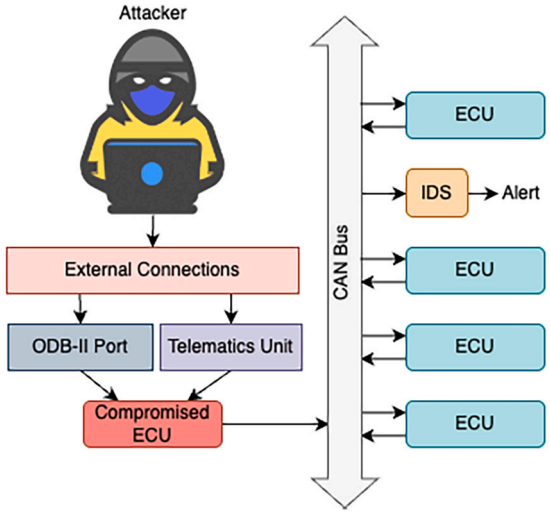


Fig. 1. An Intrusion Detection Scenario for the CAN Bus and its Protection using an IDS Structure.

manipulated CD, and that the telematics unit of a vehicle can be fully controlled remotely by calling its cellular phone number. In 2012, a BMW car was stolen by the thieves in the UK while it was parked. The thieves used a laptop and a smartphone connected to the OBD-II port to get over the anti-theft system of the vehicle, allowing them to steal the car without triggering alarms or immobilizers [8]. Furthermore, Woo et al. [9] introduced a remote attack approach using a malicious self-diagnostic application. By exploiting such an application to monitor or diagnose the vehicle's status, an attacker can gain remote control of the vehicle and execute the attack remotely. Likewise, Miller and Valasek [10] demonstrated a remote attack targeting multiple Fiat-Chrysler vehicles. It affects critical systems like braking and steering, and can be carried out without requiring modifications to the vehicle or physical interaction by the attacker. It affects critical systems like braking and steering. In 2016, Savage et al. [11], successfully controlled the brakes and windshield wipers of a Chevrolet Corvette car using a telematics control unit. In 2017, Palanca et al. [12] captured CAN frames from the bus to identify the identifier value of the ECU that they targeted. After obtaining the identifier value of a parking sensor, a denial-of-service (DoS) attack was performed. In the same year, Nie et al. [13] successfully conducted a remote attack on the Tesla Model S in both Driving and Parking modes. It has been demonstrated that the ability to gain access to the CAN Bus via wireless networks (Cellular/Wi-Fi) and place malicious CAN frames into it. Therefore, gaining access to the control unit of the vehicle allows remote manipulation of the locking, brake, and navigation systems.

Protecting vehicles from cyberattacks, as mentioned above, is essential to ensure the safety of passengers. As illustrated in Fig. 1, an Intrusion Detection System (IDS) can continuously monitor in-vehicle CAN bus traffic and detect potential attacks launched by an adversary who gains access to the bus through external connections, such as the OBD-II port or a telematics unit. Upon detecting a potential threat, the IDS triggers an alert to notify of the intrusion.

In the literature, numerous IDSs have been proposed to protect the CAN bus from cyberattacks, as discussed in detail in Section 3. However, these systems have certain limitations. Most are designed for only one or two datasets, making them less generalized to real-world scenarios where diverse driving conditions, vehicle models, and attack patterns exist. Moreover, they typically defend against only a limited range of attacks, usually around three or four types. This narrow focus makes them less effective in addressing the evolving nature of cyber threats in automotive networks. Many can only detect anomalies without identifying the specific attack type, while some also struggle with achieving high accuracy in detecting specific attacks. This lack of attack type identification reduces their usefulness for cybersecurity response teams, as different attacks may require different mitigation strategies. These challenges highlight the need for more comprehensive and adaptable IDS solutions for securing CAN bus networks.

To address the above limitations, this paper presents MetaCAN, a machine learning-based Intrusion Detection System (IDS) optimized through an adaptive hybrid metaheuristic approach for CAN bus security. This hybrid method combines particle swarm optimization (PSO) and cuckoo search (CS), leveraging their respective strengths—PSO's rapid convergence and CS's robust global search capability. Furthermore, the accuracy of the system is enhanced through a feature engineering process. The key contributions of this paper are as follows:

1. An optimized IDS for CAN Bus Security, integrating PSO and CS in a Hybrid Metaheuristic Approach. This approach combines the search spaces of both PSO and CS to effectively reduce the risk of local optima while improving convergence toward more optimal solutions. Additionally, extra features have been incorporated during the feature engineering phase to further improve the IDS's detection accuracy.
2. Evaluating the performance of the proposed IDS using three real-world datasets that encompass a range of driving scenarios and attack patterns. This comprehensive evaluation ensures the IDS is tested under realistic conditions.
3. Unlike many existing IDSs that focus on binary detection, our proposed IDS is capable of accurately identifying five distinct attack types: DoS, fuzzy, spoofing, malfunction, and replay. This multi-class detection feature broadens its practical applicability, covering a wider range of threats and offering more detailed insights into various cyberattacks.
4. The proposed IDS has been benchmarked against state-of-the-art vehicle IDSs, demonstrating superior performance. This improvement highlights the effectiveness of the hybrid metaheuristic approach and the additional features introduced in enhancing CAN bus security.

The rest of the paper is organized as follows: Section 2 provides background knowledge necessary to understand the proposed contribution. Section 3 reviews the closely related works in the field. Section 4 presents the proposed MetaCAN IDS. Section 5 describes the datasets used in this research. Section 6 analyzes and discusses the performance results. Finally, Section 7 concludes the paper outlining future research directions.

2. Background

In order to make the paper self-contained, this section provides an overview of the foundational concepts necessary to understand the relationship between the security of the CAN bus and IDS. It begins by discussing the CAN bus, a robust vehicle communication standard widely used in automotive systems for enabling efficient data exchange between electronic control units (ECUs). The discussion highlights the importance of CAN in modern vehicles and its vulnerability to cyber threats due to the absence of inherent security measures. Following this, the section introduces IDS as a critical countermeasure, outlining its role in detecting and mitigating malicious activities on the CAN bus by monitoring traffic patterns, identifying anomalies, and securing vehicular networks.

2.1. Controller Area Network (CAN)

The Controller Area Network (CAN) bus is a message-based serial communication protocol that enables electronic components within a vehicle to communicate seamlessly. Each component connects to the CAN bus through its own Electronic Control Unit (ECU). A regular CAN

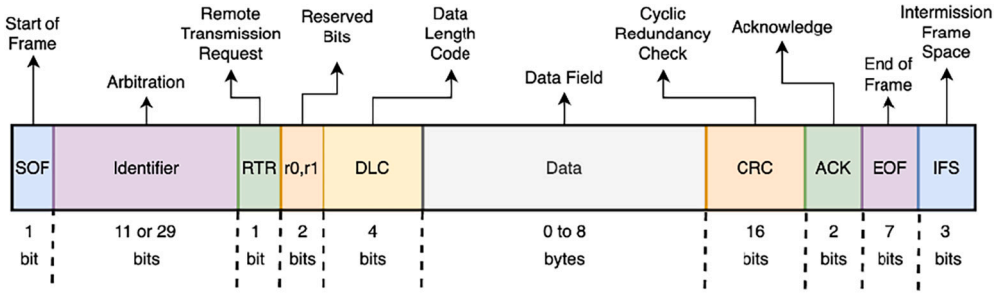


Fig. 2. A Typical CAN Frame Structure (Adapted From [5]).

in modern connected vehicles consists of 20 to 100 ECUs [14] that manage a wide range of vehicle functions such as brakes, airbag deployment, steering, gears, climate control, lighting, and engine status [15].

Originally introduced by Robert Bosch GmbH in 1980s [16], it is now a widely adopted standard for automotive applications, functioning at the data and physical link layers of the OSI model [17]. There are a few versions of the CAN bus, namely classical CAN, CAN with Flexible Data Rate (CAN FD), and CAN Extra Long (CAN XL) [18]. Each of them has distinct capabilities tailored to different application needs. The classical CAN is standardized under ISO 11898-1[1] and enables connections between ECUs at speeds up to 1 Mbps. To address growing data demands, CAN FD was introduced under ISO 11898-2[2] to support faster data transfers with speeds up to 8 Mbps. It accommodates larger data payloads for applications requiring higher bandwidth. The most recent development, CAN XL defined under ISO 11898-3,[3] is designed to meet the demands of next-generation applications, offering data transfer rates as high as 10 Gbps. This makes CAN XL a strong candidate for use in data-intensive systems. Among these options, the classical CAN bus is the most widely used because of its simplicity and affordability [19].

A typical CAN frame structure is given in Fig. 2. This frame consists of 9 fields. The **Start of Frame (SOF)** field (1 bit) marks the beginning of a message with a dominant zero, alerting all ECUs on the CAN bus. The **Identifier** field (11 or 29 bits) determines message priority, with lower values indicating higher priority. For instance, 0x20 may represent engine control, while 0x30 could indicate the brake system. This unique arbitration mechanism prevents bus collisions. The **Remote Transmission Request (RTR)** field (1 bit) differentiates between actual messages (dominant zero) and message requests (recessive one). The **Control** field (6 bits) includes a **Data Length Code (DLC)** (4 bits), specifying the data size (0–8 bytes), with 2 reserved bits typically set to zero. The **Data** field (0–8 bytes) carries the actual transmitted information, varying in length based on the message size. The **Cyclic Redundancy Check (CRC)** field (16 bits) ensures data integrity through error detection. The **Acknowledge (ACK)** field (2 bits) confirms proper message reception, prompting retransmission if an error is detected. The **End of Frame (EOF)** field (7 bits) marks the message's conclusion with a recessive bit flag. Finally, the **Intermission Frame Space (IFS)** field (3 bits) provides a standardized time gap between consecutive messages, ensuring smooth data flow.

The CAN bus communication channel relies on a pair of twisted wires designated as CAN High and CAN Low. In this protocol, two distinct states are used to represent data bits. These states are called as recessive (logical 1) and dominant (logical 0). When data is transmitted from the CAN interface, the CAN transceiver (acting as both transmitter and receiver) converts each data packet into a differential signal before sending it across the twisted pair wires. During the transmission of a recessive bit, both CAN High and CAN Low wires carry a voltage

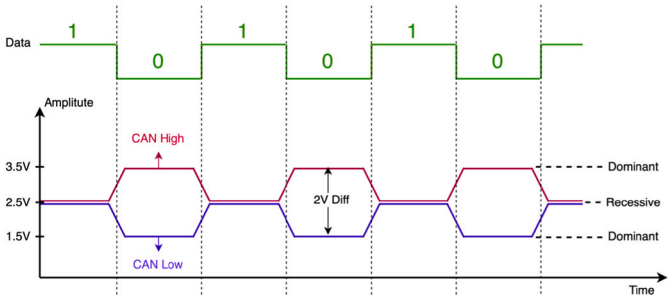


Fig. 3. Presentation of CAN Bus Differential Signal Design.

of 2.5 V. With identical voltages on each wire, the differential voltage is effectively zero. In contrast, the voltage on CAN High is increased by 1 V to 3.5 V, while the voltage on CAN Low is decreased by 1 V to 1.5 V for a dominant bit. This results in a 2 V differential between CAN High and CAN Low. It clearly helps to distinguish the dominant bit. Fig. 3 shows the voltage levels of the CAN bus. This design provides highly resilience to magnetic and electrical interference by maintaining a continuous voltage on the wires. In this way, it ensures stable and reliable communication in automotive environments [20]. This simple two-wire structure also reduces wiring costs compared to traditional point-to-point wiring.

Unlike other standard communication protocols in which each packet has specific source and destination addresses, the CAN bus lacks these address fields. Instead, it uses a broadcasting communication approach, allowing each ECU in the network to send and receive packets on the bus. This approach makes data accessible to all connected ECUs. Each ECU can request access to the bus at random intervals. To manage this, an arbitration mechanism (see Fig. 4) prioritizes accesses of ECUs. This priority is determined by comparing the arbitration IDs of the requesting ECUs. The ECU with the highest-priority ID is granted access to transmit data on the bus. As a result, ECUs responsible for critical functions, such as the engine and braking systems, typically have higher priority [21].

2.2. Vulnerabilities of the CAN bus protocol

The CAN bus protocol was modeled to be fast, resilient, and efficient, but it also brings along some vulnerabilities that attackers can exploit to gain network access and inject malicious messages for various intentions. These vulnerabilities can be assessed through the principles of Availability, Confidentiality, and Integrity [20].

- **Availability**: it indicates that authorized users can always gain access to the CAN bus. It ensures that legitimate messages are transmitted without disruption. Several attacks on availability can be carried out over the CAN bus. One of the most significant threats is a Denial-of-Service (DoS) [22]. It floods the network with fake

[1] https://www.iso.org/standard/86384.html.
[2] https://www.iso.org/standard/85120.html.
[3] https://www.iso.org/standard/36055.html.

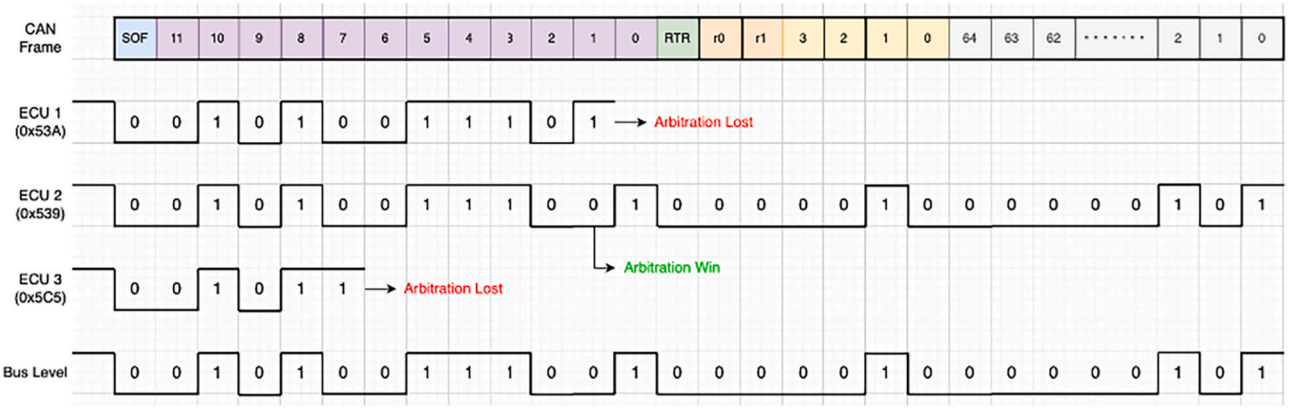


Fig. 4. CAN Bus Arbitration Mechanism.

requests, consuming memory or bandwidth and making devices or the network inaccessible.

- **Confidentiality**: it states that access to data on the CAN bus is restricted to authorized users only. However, due to the broadcast nature of the CAN bus, any transmitted message is received by all ECUs. Therefore, a targeted ECU or intruder can intercept sensitive messages, potentially leading to a privacy breach.
- **Integrity**: It indicates that a message sent from the sender ECU is received by the receiver ECU in its complete and unaltered form. The CAN bus has built-in integrity checking bits using Cyclic Redundancy Code (CRC) as shown in Fig. 2. If the checksum value of the receiver ECU does not match the value in the CRC field, it signifies a transmission error. However, a malicious device can inject faulty bits, causing errors that lead the receiver ECU to discard the transmission. Also, the CAN bus lacks an authentication mechanism. It means there is no method to confirm the integrity of the address of sender ECU.

2.3. Classifying cyberattacks targeting the CAN bus

The increasing number of electronic components in vehicles has provided high driving safety, raised comfort, and a better driving experience. Despite these electronic advancements in the automotive, vehicles have become vulnerable to various cyberattacks. These attacks can be divided into two main categories as remote access attacks and local access attacks [23,24].

- **Remote Access Attacks**: Remote access attacks are carried out without physical access to the CAN bus using various wireless networks such as Wi-Fi, Bluetooth, and 4G/5G/B5G etc. These attacks can be classified based on the range of wireless access as short and long range attacks [23]. Short range attacks involve communication technologies with limited wireless access, such as Wi-Fi, Bluetooth, remote keys, and dedicated short-range communications (DSRC). On the other hand, long range attacks target wireless communication technologies with a range greater than 1 km [25]. This involves cellular technologies such as 4G/5G/B5G and global navigation satellite system (GNSS).
- **Local Access Attacks**: Local access attacks on the CAN bus network necessitate either direct or indirect access [26]. Direct access is typically gained through the On-Board Diagnostic (OBD) port, allowing attackers to monitor or inject messages into the bus. Indirect access, on the other hand, involves connecting a compromised device, such as a smartphone, USB drive, or portable media player, through an entry point like the vehicle's multimedia system. This connection provides a pathway for attacks on the CAN network.

2.4. Attack types in CAN bus

The various cyberattacks in CAN bus are shown in Fig. 5 and explained below. These attacks can be applied both remotely and locally [14].

- **Fuzzy Attack**: Due to the lack of authentication on the CAN bus protocol, any receiver ECU connected to the bus can accept every message on the bus without verifying whether it is coming from a legitimate ECU or not [27]. An intruder can create messages with random identifiers and inject them to the bus. These lots of malicious messages are received by the various ECUs. Thus, it may lead to communication delays, cause an interruption in data dissemination, or result in the vehicle exhibiting unexpected behavior such as turning on warning lights, braking, or disabling certain functionalities.
- **DoS/Flooding Attack**: As detailed in Section 2.1, the CAN protocol relies on an arbitration mechanism to prioritize which message is sent first. This priority is determined based on the identifier value of the message (the lower identifier value has higher priority). The DoS (Denial of Service) attack can be executed by an intruder to make the bus busy by sending an excessive number of messages with the highest priority identifier (e.g. 0x000) [28]. Therefore, other ECUs cannot use the bus due to this flooding of messages. Another instance of a DoS attack occurs when the frequency of injected messages on the bus increases. In this scenario, an attacker sends messages using the identifier of a legitimate ECU. By transmitting these messages at a higher frequency than the ECU's original interval, the intruder can disrupt the sensor system.
- **Malfunction Attack**: A malfunction attack relies on data manipulation of messages with the identifier of a specific ECU. It intends to manipulate the functionality of this ECU [29]. So, the identifier of this ECU has to be extractable for a targeted vehicle. For example, an intruder might inject a message that overrides the legitimate speed sensor reading, causing the vehicle to display an incorrect speed or trigger incorrect responses. This could lead to issues such as brake malfunctions, erratic behavior of lights, or engine stalls. While DoS attacks focus on overwhelming the system to block communication, malfunction attacks are more targeted to alter the behavior of a specific ECU.
- **Replay Attack**: A replay attack is a type of cyberattack where valid data transmission is maliciously or fraudulently repeated [4]. In this attack type, an intruder retransmits the same message of an existing ECU. As in a malfunction attack, the identifier of the ECU must be extractable for the targeted vehicle to retransmit the same message. In other words, the primary goal of this attack type is to retransmit data instead of generating new data. This allows the attacker to impersonate an existing ECU. In this way, an intruder might intercept

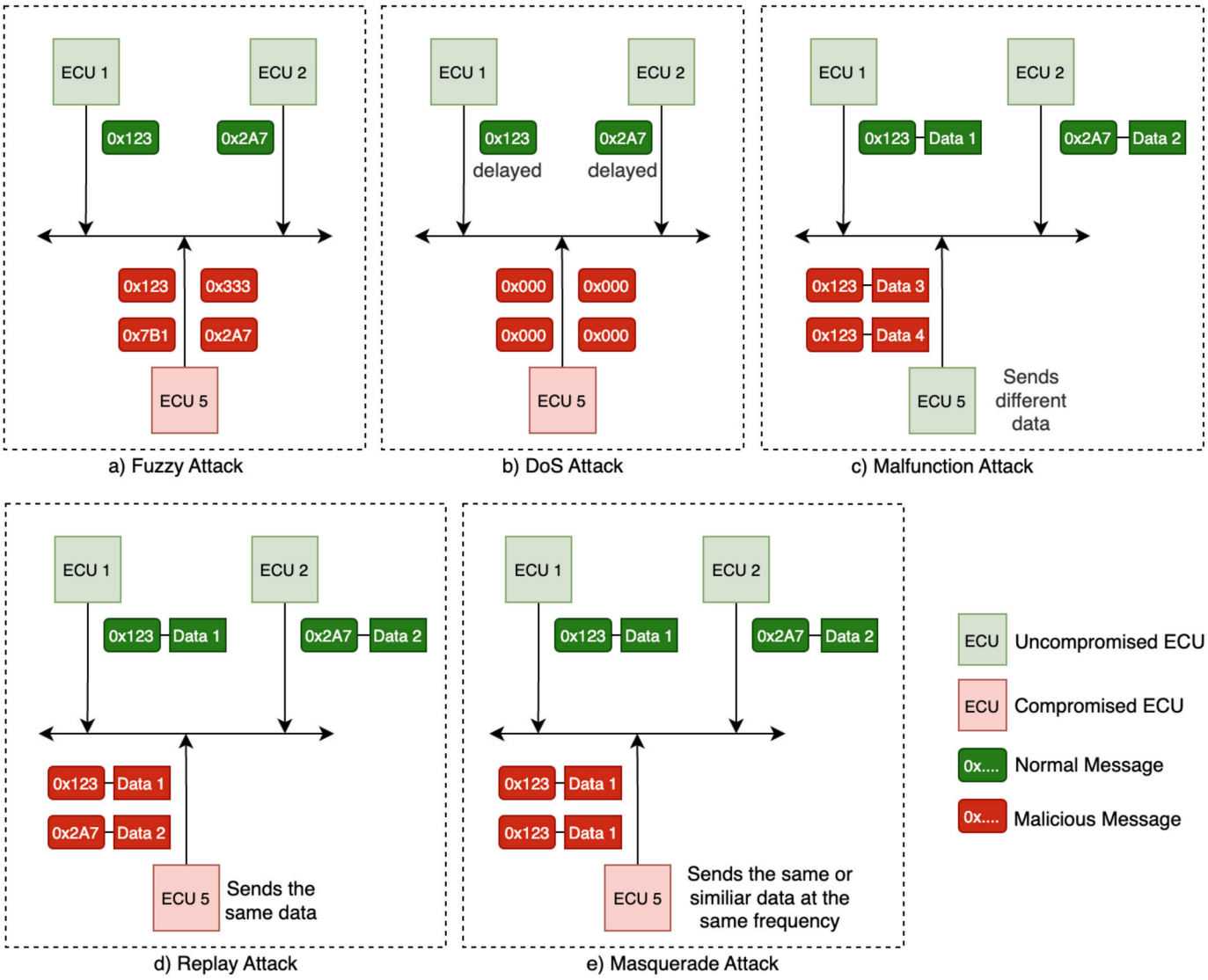


Fig. 5. Different Types of Cyberattacks in CAN Bus, a) Fuzzy Attack, b) DoS Attack, c) Malfunction Attack, d) Replay Attack, and e) Masquerade Attack.

a legitimate CAN bus message such as commanding to unlock doors or starting the engine and replay it to gain unauthorized access to the vehicle.

- **Masquerade/Spoofing/Impersonation Attack**: The aim of the masquerade attack is to manipulate the actions of an ECU without revealing that it has been compromised. An intruder needs to be two ECUs for this attack type [30]. For example, first ECU (with the identifier 0xAA) observes the messages of second ECU (with the identifier 0xBB) to learn the identifier and frequency values of it. After the first ECU learned this information of second ECU, an intruder prevents the second ECU from sending messages to bus and enables the first ECU to send malicious messages with the identifier value of the second ECU at the same frequency. Although the traffic on the bus appears unchanged, the messages are now being sent by first ECU instead of second ECU, granting the intruder control over the data within the messages.

2.5. Intrusion Detection System (IDS)

An IDS is a critical security mechanism designed to track and analyze network traffic for signs of malicious activity or unauthorized access [31]. It detects potential threats by analyzing data packets, comparing them against predefined rules or known attack signatures, and identifying abnormal patterns of behavior. An IDS built for IoT-based smart environments must function efficiently under challenging conditions such as limited processing capacity, the need for rapid response times, and the ability to handle high volumes of data [32]. There are many IDSs designed to secure the CAN bus. These systems can be grouped into two main categories as signature-based and anomaly-based IDSs [33].

2.5.1. Signature-based IDSs

Signature-based IDSs operate by comparing incoming traffic with known attack signatures. These signatures are predefined patterns that represent specific attack behaviors. When the system identifies a match between observed CAN bus traffic and a stored signature, it triggers an alert. This method is highly effective at detecting known threats but fails to identify new or previously unseen attacks. To remain effective, it requires frequent updates to its signature database [34].

2.5.2. Anomaly-based IDSs

Anomaly-based IDSs focus on detecting deviations from normal operational behavior rather than matching signatures. These systems first establish a baseline of normal traffic behavior, such as typical message

frequency, data sizes, and communication patterns between ECUs. Any significant deviation, such as irregular message timings or unexpected data patterns, is flagged as a potential anomaly. Although this approach may have lower accuracy compared to signature-based IDSs, it can detect previously unknown or emerging attacks since it does not depend on predefined signatures [24].

Anomaly-based IDSs can be further classified into several subclasses such as specification-based [35], time/frequency-based [36], physical system characteristic [37], and feature-based. The most popular ones, feature-based IDSs [38], extract specific characteristics (features) from CAN bus traffic and use them for anomaly detection. These features might include message IDs, data payloads, timing, or statistical properties of the traffic. In general, machine learning methods are applied to analyze these features to indicate anomalies. The advantage of machine learning-based IDS in CAN lies in its ability to detect unknown and previously unseen attacks by learning the patterns of CAN bus traffic. This makes machine learning-based IDSs highly effective in identifying sophisticated cyberattacks [39].

3. Related work

This section primarily reviews various IDS techniques developed to enhance CAN bus security. The studies discussed explore diverse approaches, including deep learning, ensemble methods, transfer learning, and graph-based models, among others. Table 1 provides a summary comparison of these IDS techniques, which are further elaborated in the subsequent discussion. Although these IDS models demonstrate promising results in detecting cyberattacks on the CAN bus, they also face limitations such as data set dependency, restricted attack type coverage, and generalizability challenges.

Hossain et al. [40] proposed long short-term memory (LSTM) based IDS for CAN bus security. The IDS was tested with two different datasets: one of them is created using data collected from a Toyota hybrid car and the other one is created using data collected from three different cars. The study also explores hyperparameter tuning and examines how variations in hyperparameter values affect detection accuracy and detection rates. Using the selected hyperparameter values, the results demonstrate that the classifier effectively detects CAN bus network attacks for both datasets.

Javed et al. [41] proposed an intrusion detection approach named CANintelliIDS for the CAN bus. It combines convolutional neural networks (CNN) with attention-based gated recurrent units (GRU). This hybrid model is designed to detect both single and mixed intrusion attacks on the CAN bus. Evaluated on OTIDS dataset, the combined model demonstrated improved performance over existing methods.

Mehedi et al. [42] proposed a deep transfer learning based LeNet model for detecting anomalies in CAN networks by focusing on effective attribute selection to identify malicious messages. The data preprocessing step is carried out through three key steps such as removing irrelevant features, converting non-numeric attributes into numerical formats, and eliminating outliers. The model demonstrated robust performance to classify normal and abnormal activities. Thus underscores the potential of transfer learning in enhancing in-vehicle IDSs.

Anjum et al. [43] proposed an anomaly detection approach utilizing the extreme gradient boosting (XGBoost) algorithm to identify unexpected occurrences in CAN bus. The method incorporates both CAN IDs and data payloads to predict anomalies effectively. To enhance performance, the study employed parameter tuning with grid search. Experimantal results revealed that XGBoost, known for its computational efficiency and speed, outperformed other gradient boosting techniques.

Alalwany et al. [44] three ensemble methods which are voting, stacking, and bagging for intrusion detection utilizing eight supervised machine learning models. This framework leverages the advantages of multiple learning mechanisms to enhance prediction reliability and reduce classification errors. Ensemble classifiers demonstrated superior performance by combining weighted predictions from individual classifiers.

Driss et al. [45] introduced an IDS which integrates a group of gated recurrent units (GRUs) with a random forest classifier to enhance attack detection accuracy for cyberattack detection in vehicular networks. In this approach, the features retrieved from the GRU are then fed into the random forest. By leveraging on-device training and sharing computational capabilities, the IDS ensures efficient and privacy-preserving detection of cyberattacks. Experimental results demonstrate the effectiveness of the proposed method in detection of cyber threats across multiple performance metrics.

Caivano et al. [46] introduced a random forest based IDS to detect attack injections on a CAN bus. The study utilized the Car-Hacking Dataset for binary intrusion detection and extended the analysis to multiclassification by creating a concatenated dataset from the Synthetic Automotive Hacking Dataset and the Survival Analysis Dataset. To ensure uniform feature scaling, all features were normalized to a range between 0 and 1 using Min Max Scaler. Hyperparameter optimization of the random forest was conducted through a comprehensive grid search to evaluate all possible parameter combinations.

Cobilean et al. [47] proposed a transformer neural network based IDS to predict anomalous behavior within CAN bus communication by simultaneously analyzing both CAN ID sequences and message payload values. This approach eliminates the need for labeled data and leverages a fully self-supervised learning mechanism. The transformer network predicts the next sequence of communication and anomalies are identified by comparing the predicted and actual sequences. The method demonstrates the advantages of transformer architectures for enhanced anomaly detection for the Survival Analysis Dataset.

Similarly, Mavikumbure et al. [48] proposed a Domain Adversarial training-based AutoEncoder (DAdAE) for unsupervised anomaly detection in CAN bus communication. Since this approach is based on unsupervised learning, it does not require labeled data and makes it suitable for scenarios with limited datasets. The proposed DAdAE model was evaluated on the Survival Analysis Dataset and the experimental results demonstrate that it significantly enhances the overall F1-score compared to other unsupervised models.

Park et al. [49] represented a graph-based IDS named G-IDS and designed to enhance the security of the CAN protocol in vehicular networks. It integrates a threshold-based attack classifier with a machine learning-based classifier to address the limitations of existing IDS approaches such as reliance on a large number of CAN messages and the inability to classify attack types effectively. Graph-based features are extracted from the constructed graph and utilized to determine the threshold for the binary threshold-based classifier as well as to train the multi-class machine learning classifier.

Al-Aql et al. [50] introduced a long short-term memory (LSTM) based IDS for intrusion detection in vehicle networks. First, feature extraction was performed using recurrent neural networks (RNN), and the extracted features were provided as input to the LSTM to capture long-term dependencies. This approach demonstrated superior performance with 93% accuracy.

Khan et al. [51] proposed an ensemble model named DivaCAN to effectively detect cyberattacks on the CAN bus. This method strategically balances high classification performance with practical time complexity. Thus makes it a robust and efficient solution for intrusion detection in automotive networks.

Xiao et al. [52] have proposed a graph node attention network (GNAT) method to extract context-dependent features from CAN frame sequences. This method converts time-continuous frames into graphs by mapping ECUs based on their arbitration fields and connecting them through time-adjacent edges. Then, extracted features are used to train a random forest classifier for detecting malicious messages. Experimental results demonstrate that GNAT improves generalization and detection accuracy of the classifier for CAN bus security.

Table 1
Comparison of the Existing IDS Techniques for Securing the CAN Bus.

Author and Year	Model	Attack Types	Data Source Type	Dataset	Strengths (+) and Limitations (-)
Hossain et el. 2020 [40]	LSTM	DoS, Fuzzy, Spoofing, Malfunction	Real	Survival Analysis, NAIST CAN	- No feature engineering conducted. - No cross-validation performed.
Javed et al. 2021 [41]	CNN+GRU	DoS, Fuzzy, Spoofing	Real	OTIDS	- Limited attack type. - No feature engineering conducted. - Cross validation performed.
Mehedi et al. 2021 [42]	LeNet	Binary Classification	Real	Car Hacking: Attack & Defense Challenge 2020	- No feature engineering conducted. - No cross validation performed.
Anjum et al. 2022 [43]	XGBoost	DoS, Fuzzy, Malfunction	Real	Survival Analysis	+ Gird search conducted to optimize the model. + Cross validation performed. - Limited attack type. - No feature engineering conducted.
Alalwany et al. 2022 [44]	Ensemble Model	Binary Classification	Real	Car Hacking: Attack & Defense Challenge 2020	+ Random search conducted to optimize the model. - No feature engineering conducted. - No cross validation performed.
Driss et al. 2022 [45]	Random Forest	Binary Classification	Real	Car Hacking: Attack & Defense Challenge 2020	+ Feature engineering conducted. - No cross validation performed.
Caivano et al. 2023 [46]	Random Forest	DoS, Fuzzy, Malfunction, Fabrication	Real and Synthetic	Car-Hacking, Concatenated Dataset (Synthetic Automotive Dataset+Survival Analysis)	+ Gird search conducted to optimize the model. + Cross validation performed. - No feature engineering conducted. - Results comparison is limited.
Cobilean et al. 2023 [47]	Transformer Neural Network	Binary Classification	Real	Survival Analysis	+ No need labeled data for training. - Binary detection only. - No feature engineering conducted. - No cross validation performed.
Mavikumbure et al. 2023 [48]	Domain Adversarial Training Based AutoEncoder	DoS, Fuzzy, Malfunction	Real	Survival Analysis	+ Feature engineering conducted. - Limited attack type. - No cross validation performed.
Park et al. 2023 [49]	Random Forest	DoS, Fuzzy, Malfunction, Spoofing	Real	Car Hacking, Survival Analysis, Car Hacking: Attack & Defense Challenge 2020	+ Timing analysis investigated. + Feature engineering conducted. - No cross validation performed.
Al-Aql et al. 2024 [50]	LSTM	DoS, Fuzzy, Spoofing	Real	OTIDS	+ Feature engineering conducted. - Limited attack type. - No cross validation performed.
Khan et al. 2024 [51]	Ensemble Model	DoS, Fuzzy, Spoofing	Real	OTIDS	+ Timing analysis investigated. - Limited attack type. - No feature engineering conducted. - No cross-validation
Xiao et al. 2024 [52]	Random Forest	DoS, Fuzzy, Spoofing, Malfunction	Real	Car-Hacking, Survival Analysis	+ Feature engineering conducted. - No cross-validation performed. - Complex graph conversion algorithm.
MetaCAN	**XGBoost**	**DoS, Fuzzy, Spoofing, Malfunction, Replay**	**Real**	**Survival Analysis, Car Hacking: Attack & Defense Challenge 2020, OTIDS**	**+ Feature engineering conducted.** **+ Cross-validation performed.** **+ Metaheuristics conducted to optimize the model.** **+ Covers more diverse attack types.**

These IDSs can secure the CAN bus. However, they also come with certain limitations. A significant drawback is that most of these IDSs are tailored to work with only one or two specific datasets, limiting their generalizability to other datasets or real-world scenarios. As a result, they cannot be considered fully generalized models. Furthermore, their effectiveness in securing the CAN bus is restricted to a limited range of attack types. They typically address only three or four variations of attacks. Many of these IDSs are designed merely for anomaly detection and lack the capability to identify or classify the specific type of attack. Additionally, some models suffer from low accuracy when it comes to

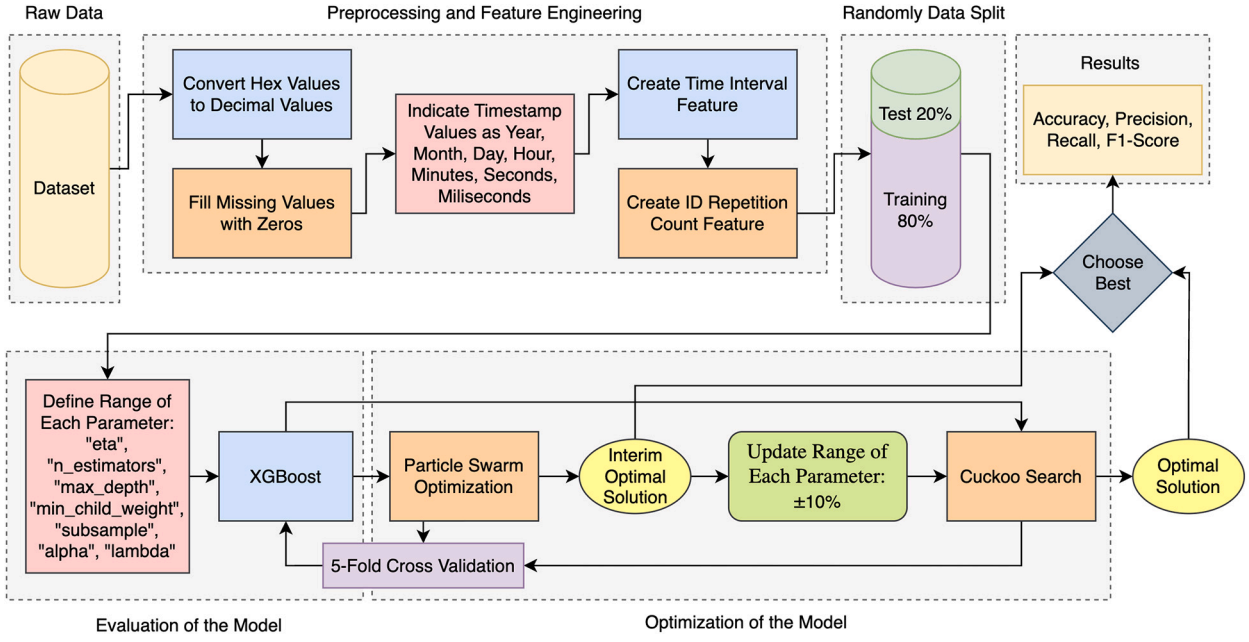


Fig. 6. Flowchart of the Proposed MetaCAN IDS.

detecting certain types of attacks, reducing their reliability in critical applications.

There are also IDSs designed using simulated CAN bus datasets [53]. However, the datasets mentioned earlier are considered more realistic as they are collected from real vehicles. Similarly, the proposed IDS in this study focuses on real-world datasets to enhance its practicality and applicability in real-world scenarios.

The above review presents a range of IDSs for securing the CAN bus in vehicular networks, exploiting diverse machine learning and deep learning techniques such as LSTM, CNN-GRU hybrids, XGBoost, and transformer networks. While these approaches demonstrate promising detection accuracy and performance enhancements, their dependence on specific datasets remains a key limitation, restricting their generalization to real-world scenarios. Moreover, many IDSs focus on anomaly detection without effectively classifying attack types, and some suffer from low accuracy for certain threats. Therefore, while these IDSs contribute to the security of the CAN bus, challenges remain to enhance their adaptability and robustness.

4. Proposed IDS: the MetaCAN framework

Machine learning-based IDSs have been widely developed to enhance the security of CAN bus networks. Some approaches utilize machine learning models without any optimization process, while others incorporate preliminary optimization method such as Grid search [54]. Grid search often provides improved results compared to directly applying machine learning models without any optimization. However, its effectiveness is constrained by its limited ability to explore a broad range of parameter values. In contrast, metaheuristic optimization techniques offer greater flexibility and can identify optimal parameter configurations more effectively [55]. Furthermore, hybrid metaheuristic approaches can enhance model performance by achieving superior hyperparameter tuning [56,57]. Consequently, this study employs a hybrid metaheuristic approach to optimize the hyperparameters of the machine learning model.

4.1. Methodology

The overall process of the proposed IDS is illustrated in Fig. 6. The methodology consists of the following steps:

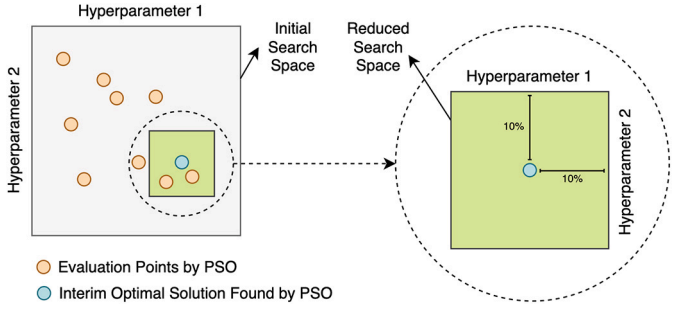


Fig. 7. Determination of Reduced Search Space for a 2D Space.

Preprocessing and Feature Engineering: (i) Convert all hexadecimal values to decimal values; (ii) Fill missing values with zeros; (iii) Split the timestamp feature into new features: year, month, day, hour, minutes, seconds, and milliseconds; and (iv) Create additional features, such as the time interval and ID repetition count.

Data Splitting: The data set is divided into training and test sets to facilitate the evaluation of the model.

Hyperparameter Boundaries Definition: The upper and lower bounds for hyperparameters are defined to guide the optimization process.

Model Training: The XGBoost classifier is trained using 5-fold cross-validation to ensure generalization and robustness.

Hybrid Optimization: The XGBoost classifier undergoes optimization using a hybrid algorithm that combines PSO and CS techniques. In this hybrid approach, the initial search space is first narrowed using the solution found by PSO. Then, CS is evaluated within this reduced search space to increase the probability of finding the best optimal solution for CS. This process is detailed in Fig. 7 for a 2D space. On the other hand, there is no guarantee that the best solution will always be within this reduced search space. In such cases, MetaCAN employs its adaptive feature to utilize the result obtained from PSO.

Performance Evaluation: The optimized model is assessed using the following performance metrics: (i) Precision; (ii) Recall; (iii) F1-score; and (iv) Accuracy.

The remaining part of this section is organized as follows: First, the XGBoost method is described. Next, the standard PSO and CS meta-

heuristic optimization algorithms are explained. Finally, the proposed hybrid IDS, which integrates PSO and CS, is presented.

4.2. Extreme Gradient Boosting (XGBoost)

Extreme Gradient Boosting (XGBoost) is an extremely efficient and scalable ensemble learning technique presented by Tianqi Chen and Carlos Guestrin [58]. It implements gradient-boosted decision trees and was developed to optimize both performance and speed by addressing the challenges such as memory usage, consumption time, and imbalanced data [59]. The algorithm works by sequentially building decision trees. Each new tree is modeled to correct the errors made by the previous ones. Therefore, the new tree is constructed to eliminate the residual errors from the previous tree. This iterative approach allows XGBoost to combine multiple weak learners effectively. This leads to the production of a strong and accurate predictive model. Also, its robust design includes features such as parallel and distributed computing [60]. This capability has made it a popular choice for solving both regression and classification problems in various fields.

For a better explanation of the boosting approach in XGBoost, consider for a given training dataset: $D_{train} = \{(x_i, y_i) : i = 1, \ldots, N \text{ and } x_i \in R^M \text{ and } y_i \in R\}$ where M indicates the features and N represents the number of observations. The output of the tree boosting approach ($\ddot{y}_i$) can be defined as follows [61]:

$$\ddot{y}_i = \sum_{t=1}^{T} f_t(x_i), \quad f_t \in T \tag{1}$$

where T is the number of the trees and $f_t(x_i)$ represents the predicted value provided by the t_{th} tree for the i_{th} observation. The set of functions f_t is learned by minimizing the objective function (F_{obj}) which can be obtained as follows:

$$F_{obj} = \sum_{i=1}^{N} F_{loss}(\ddot{y}_i, y_i) + \sum_{t=1}^{T} \Omega(f_t) \tag{2}$$

where F_{loss} indicates the training loss function and it evaluates the difference between the target value y_i and the predicted value $\ddot{y}_i$; and Ω indicates the regularization term and it is used to evaluate the complexity of the tree f_t.

The regularization term (Ω) is calculated as follows:

$$\Omega(f_t) = \alpha L + \frac{1}{2}\beta\omega^2 \tag{3}$$

where α and β are the degrees of regularization, L is the number of the leaves, and ω is the vector of values attributed to each leaf.

The objective function in Equation (2) for the tree boosting approach cannot be optimized using conventional methods in Euclidean space [60]. However, gradient tree boosting enhances this process by training the model incrementally, with predictions at the k_{th} iteration expressed as:

$$\ddot{y}^{(k)} = \ddot{y}^{(k-1)} + f_k(x) \tag{4}$$

Therefore, the objective function at the k_{th} iteration is modified as follows:

$$F_{obj}^{(k)} = \sum_{i=1}^{N} F_{loss}\left(y_i, \ddot{y}_i^{(k-1)} + f_k(x_i)\right) + \Omega(f_k) \tag{5}$$

XGBoost employs a second-order Taylor expansion to approximate Equation (5). This reformulation allows the objective function at iteration k to be expressed as:

$$F_{obj}^{(k)} = \sum_{i=1}^{N} \left[F_{loss}(y_i, \ddot{y}_i^{(k-1)}) + g_i f_k(x_i) + \frac{1}{2} h_i f_k(x_i)^2\right] + \Omega(f_k) \tag{6}$$

where $g_i = \partial_{\ddot{y}(k-1)} F_{loss}(y_i, \ddot{y}_i^{(k-1)})$ and $h_i = \partial^2_{\ddot{y}(k-1)} F_{loss}(y_i, \ddot{y}_i^{(k-1)})$ the first and second order gradient statistics on the loss function, respectively.

To achieve the best performance with XGBoost, its hyperparameters must be carefully tuned. It mainly includes the following key hyperparameters:

- **n_estimators:** It states the number of decision trees in the model,
- **min_child_weight:** It sets the minimum sum of instance weights for creating a leaf node,
- **eta:** It indicates the learning rate and controls the step size for boosting iterations,
- **alpha:** It specifies the regularization term L1 applied to leaf weights,
- **lambda:** It refers to the regularization term L2 on leaf weights,
- **subsample:** It indicates the fraction of training samples used in each boosting round,
- **max_depth:** It represents the maximum depth of each decision tree.

4.3. Metaheuristics

Metaheuristics are high-level optimization frameworks designed to solve complex optimization problems that are challenging for conventional methods. These techniques explore the search space efficiently by combining randomness and heuristic-based rules [62]. Two of these metaheuristic optimization methods which are standard PSO and standard CS are explained below.

4.3.1. Particle Swarm Optimization (PSO)

Eberhart and Kennedy developed the PSO algorithm by drawing inspiration from the organizational behaviors of birds and fish such as flocking and schooling movements [63]. In PSO, each bird or fish is referred to as a particle. The search space is the area containing all potential solutions, including the best solution. Particles move within this search space based on the objective function to find the best solution. The movement of each particle is determined by both its own experience and the experiences of its neighbors.

In the first step of PSO, the positions of the particles are initialized randomly. Similarly, the velocity of each particle is also initialized randomly. In each iteration, the new velocity of each particle is calculated based on its best-known position, current velocity and the best-known positions of its neighbors. The positions of the particles are then updated until the stopping criterion is satisfied [64].

The velocity and position of each particle i are updated in each iteration of the PSO as follows [65]:

$$v_i(t+1) = wv_i(t) + c_1 r_1 (P_i^{Best}(t) - x_i(t)) + c_2 r_2 (G^{Best}(t) - x_i(t)) \tag{7}$$

$$x_i(t+1) = x_i(t) + v_i(t+1) \tag{8}$$

where i refers a particle in the swarm; t is the iteration index; w is the inertia weight factor; x_i and v_i are the position and velocity of the particle i, respectively; $c1$ and $c2$ are the positive constants which are called acceleration coefficients; $r1$ and $r2$ are the random values uniformly distributed with in the range [0,1]; and P_i^{Best} and G^{Best} are the best position found by particle i and best position found by the whole swarm, respectively.

4.3.2. Cuckoo Search (CS)

The CS algorithm is a nature-inspired optimization method introduced by Xin-She Yang and Suash Deb [66]. It draws inspiration from the unique and aggressive reproduction strategies of certain cuckoo bird species such as Guira and Ani [67].

The algorithm mimics the parasitic nesting behavior of cuckoos, where they exploit the nests of other birds to increase their offspring's survival chances. This behavior is called brood parasitism and includes laying eggs in host nests to synchronize incubation. It also involves the early hatching of cuckoo chicks that push host eggs out and the mimicry of egg patterns to avoid detection by the host bird. If detected, host birds may discard the foreign egg or abandon the nest. The CS uses

these behaviors to enhance optimization by replacing weaker solutions with potentially superior ones [68]. Furthermore, the CS employs Lévy flights that are inspired by observations of foraging patterns in animals. These are random walks with long steps interspersed with shorter, more abrupt changes in direction. This natural behavior has been mathematically modeled using the Lévy distribution. Observed in the foraging patterns of animals, Lévy flights allow efficient exploration, enhanced convergence speed and optimized performance [68].

In CS algorithm, each solution to the optimization problem is presented by a nest, and each nest contains an egg, which represents a candidate solution. This optimization algorithm fundamentally relies on three key principles [69]:

1. Each cuckoo randomly chooses a nest and lays a single egg in it;
2. The best nests with high-quality eggs will be carried forward to the next generations; and
3. The number of available host nests is fixed, and the probability of discovering the foreign egg by a host bird is denoted by $P_a \epsilon [0,1]$. If the host bird detects the egg, it can either discard it or abandon the nest entirely to build a new one in a new location.

CS algorithm employs a balanced combination of a global explorative random walk and a local random walk. The balance between them is governed by a switching P_a parameter. The local random walk and the global random walk can be defined as follows [67], respectively:

$$x_m^{(t+1)} = x_m^{(t)} + as \circledast H(P_a - \varphi) \circledast (x_n^{(t)} - x_k^{(t)}) \tag{9}$$

$$x_m^{(t+1)} = x_m^{(t)} + aL(s,\lambda) \tag{10}$$

where φ is a random number sampled from uniform distribution; s is the step size; $\circledast$ is the entry-wise multiplication; $x_n^{(t)}$ and $x_k^{(t)}$ are two different solutions chosen randomly by random permutation; a is the scaling factor and its value is greater than 0; $H(u)$ is the Heaviside function; λ is a parameter representing the mean or expectation of the event's occurrence within a unit interval; and L represents Lévy flight distribution. Here, a significant fraction of the new solutions should be produced using far-field randomization, ensuring their positions are sufficiently distant from the current best solution. This strategy helps to avoid the system being trapped in a local optimum [70].

4.4. Hybrid metaheuristic approach

Metaheuristics cannot guarantee finding the global optimum solution (best solution) due to their stochastic nature. However, they focus on achieving high-quality solutions (near-optimal solutions) within a reasonable computation time [62]. Combining existing algorithms is an approach to enhance the likelihood of achieving a global optimum solution [71].

PSO is extensively employed in hybrid methods thanks to its rapid convergence speed and simplicity [72]. Some studies indicate that while PSO can quickly converge to the local best solution, it may not always find the global best solution due to fluctuations in local search operations [73,74]. It has also been shown that CS can search for the global optimum thanks to its Lévy flight process, but it suffers from a slow convergence rate [62,72]. To address these issues, a hybrid algorithm combining PSO and CS is proposed. In this hybrid model, PSO may prematurely converge to a local optimum, while CS can usually converge to the global optimum. Therefore, it combines the fast convergence characteristic of the PSO with the effective global search ability of CS.

The pseudo code of the proposed hybrid algorithm is given in Algorithm 1. This hybrid algorithm is used for hyperparameter optimization to find the best hyperparameter values for the XGBoost classifier. This combined model operates in a sequential manner. First, the PSO algorithm performs the optimization process. Then, the upper and lower bounds of each hyper parameter are redefined by using the optimal hyperparameter values obtained as a result of PSO process. This redefinition is performed by using Equation (11) and Equation (12). Subsequently, using these redefined bounds, the CS algorithm continues the optimization process. This approach ensures that the CS algorithm searches for solutions within a much more confined space.

$$LB_r = \begin{cases} max(LB_i, floor(G^{Best} - 0.1 \cdot G^{Best}), & \text{if } p \text{ is an integer} \\ max(LB_i, (G^{Best} - 0.1 \cdot G^{Best}), & \text{if } p \text{ is a float} \end{cases} \tag{11}$$

$$UB_r = \begin{cases} min(UB_i, ceil(G^{Best} + 0.1 \cdot G^{Best}), & \text{if } p \text{ is an integer} \\ min(UB_i, (G^{Best} + 0.1 \cdot G^{Best}), & \text{if } p \text{ is a float} \end{cases} \tag{12}$$

where p is the hyperparameter; LB_r and UB_r are the reduced lower and upper bounds of the hyperparameter p of the classifier, respectively; G^{Best} represents the best hyperparameter value obtained by the PSO for the hyperparameter p.

Algorithm 1 The Pseudo Code of the Proposed Hybrid IDS.

Set Parameters:
Define the initial upper (UB_i) and lower bounds (LB_i) of hyperparameters
Define the objective function f
Define parameters of both metaheuristic algorithms
Generate the initial population of H host nests x_m, where $m = 1, 2, \ldots, H$
Initialize the Swarm of PSO Randomly:
for $i = 1$ to swarm size **do**
    Generate V_i and X_i randomly
    Evaluate the fitness of X_i, denoting it as $f(X_i)$
    Set $P_i^{Best} \leftarrow X_i$
    Set $f(P_i^{Best}) \leftarrow f(X_i)$
end for
Set $G^{Best} \leftarrow P_l^{Best}$, where l is the particle with the best fitness
Set $f(G^{Best}) = \min f(P_i^{Best})$
for $i = 1$ to swarm size **do**
    if $f(P_i^{Best}) < f(G^{Best})$ **then**
        $G^{Best} \leftarrow P_i^{Best}$
        $f(G^{Best}) \leftarrow f(P_i^{Best})$
    end if
end for
Optimization with PSO:
$t \leftarrow 1$
while $t \leq$ maximum number of iterations **do**
    for $i = 1$ to swarm size **do**
        Randomly generate r_1 and r_2
        Evaluate velocity $v_i(t+1)$ of particle i
        Update position $x_i(t+1)$ of particle i
        Update Personal Best:
        if $f(x_i(t+1)) < f(P_i^{Best})$ **then** ▷ If new position is better than the previous personal best
            $P_i^{Best} \leftarrow x_i(t+1)$ ▷ Set the new position as personal best
            $f(P_i^{Best}) \leftarrow f(x_i(t+1))$ ▷ Update the fitness of the new personal best
        end if
        Update Global Best:
        if $f(P_i^{Best}) < f(G^{Best})$ **then** ▷ If personal best is better than the global best
            $G^{Best} \leftarrow P_i^{Best}$ ▷ Update global best
            $f(G^{Best}) \leftarrow f(P_i^{Best})$ ▷ Update fitness of the global best
        end if
    end for
    $t \leftarrow t + 1$
end while
return interim optimal solution G^{Best}
Update Bounds of Each Parameter (p):
if p is integer **then**
    $LB_r = max(LB_i, floor(G^{Best} - 0.1 \cdot G^{Best})$
    $UB_r = min(UB_i, ceil(G^{Best} + 0.1 \cdot G^{Best})$

else
  $LB_r = max(LB_i, (G^{Best} - 0.1 \cdot G^{Best})$
  $UB_r = min(UB_i, (G^{Best} + 0.1 \cdot G^{Best})$
end if
Optimization with CS:
$t \leftarrow 1$
while $t \leq$ maximum number of iterations **do**
  Get a cuckoo (say m) randomly by Lévy distribution
  Evaluate its quality/fitness F_m ▷ Calculate the fitness of the cuckoo solution
  Choose a nest among H (say n) randomly ▷ Choose a random nest for comparison
  Evaluate its quality/fitness F_n ▷ Evaluate the fitness of the nest solution
  if $F_m > F_n$ **then** ▷ If the cuckoo is better than the nest
    Replace n with the new solution
  end if
  A fraction (P_a) of worse nests are abandoned, and new ones are built via Lévy flights
  Keep the best solutions (or nests with quality solutions)
  Rank solutions and find the current best
  $t \leftarrow t + 1$
end while
return the optimal solution C^{Best}
Choosing Best Solution:
if $C^{Best} \geq G^{Best}$ **then**
  return C^{Best} as a final solution
else
  return G^{Best} as the final solution
end if

5. Datasets

There are many CAN bus datasets publicly available for researchers to analyze and design IDSs [46,53,75,76]. The creation of these datasets can differ significantly based on the source and method used. These datasets are typically generated through one of three approaches: simulation, synthetic generation, or collection from real vehicles.

Simulated datasets are generated using software-based tools or simulators that emulate the behavior of a CAN bus network. Moreover, synthetic datasets are created by combining real CAN data with artificially generated data to simulate specific scenarios or attacks. Finally, real-world datasets are collected directly from vehicles by logging CAN traffic typically through the OBD-II port during real-world operations. Among these, real-world datasets are highly realistic thanks to accurately reflecting actual vehicle behavior and traffic patterns. They provide a reliable baseline for testing IDS performance in real-world scenarios.

Therefore, real-world datasets which are OTIDS [75], Survival Analysis [76], and Car Hacking: Attack & Defense Challenge 2020 [77,78] are used in this study to design and evaluate the proposed MetaCAN IDS. These datasets were chosen because they are derived from real-world environments, cover diverse attack types, represent various scenarios such as stationary and driving conditions, and were collected using different car brands.

An overview of the CAN bus datasets used in this study is presented in Table 2. Survival Analysis dataset has three subsets created by collecting data from three different cars which are Hyundai YF Sonata, Kia Soul, and Chevrolet Spark. Each of them contains the same type of attacks: flooding, fuzzy, and malfunction. While Sonata subset includes 535,041 messages, Soul and Spark subsets include 797,843 and 402,956 messages, respectively. Similarly, Car Hacking: Attack & Defense Challenge 2020 has three subsets called Preliminary Training, Preliminary Submission, and Final Submission. All of them are created using the same car which is Hyundai Avante CN7. They include flooding, spoofing, replay, fuzzy attacks. While Preliminary Training subset includes 3,625,613 messages, Preliminary Submission and Final Submission include 3,752,046 and 1,270,310 messages, respectively. The dataset was utilized during the Car Hacking: Attack & Defense Challenge 2020 event in South Korea, enabling researchers and participants to evaluate the performance of their IDSs in real-world attack scenarios. Moreover, OTIDS dataset is created by collecting data from a Kia Soul car. It includes three different attacks which are DoS, fuzzy, and impersonation. It contains a total of 4,613,909 messages, including 2,369,868 attack-free and 2,244,041 attack messages.

Each dataset contains the following fields: Timestamp, CAN ID, DLC, Data, and Class. Among these, the Timestamp, CAN ID, DLC, and Data fields serve as input features for machine learning models, while the Class field represents the output label or category assigned to each message. For example, normal messages are labeled as Normal, while malicious messages are labeled according to their attack type, such as Fuzzy, DoS, or Impersonation. A few examples from OTIDS dataset are provided in Table 3.

The use of multiple datasets in our evaluation is intentional to reflect realistic and varied conditions. Specifically, the datasets were collected from different vehicle brands, including Hyundai YF Sonata, Kia Soul, Chevrolet Spark, and Hyundai Avante CN7. Each of these vehicles encodes the data field of CAN frames differently due to manufacturer-specific standards and communication protocols. This diversity introduces variability in both normal and attack traffic, making the detection task more challenging and thereby testing the robustness of the proposed MetaCAN model.

Moreover, the datasets contain a wide range of attack types such as DoS, fuzzy, spoofing, malfunction, and replay attacks. The variation in attack execution and data structure across vehicle types enhances the exposure of the model to heterogeneous scenarios. As a result, this setup not only increases the realism of the experimental environment but also supports the broader applicability and generalizability of our intrusion detection system in real-world automotive environments.

5.1. Data pre-processing and feature engineering

The dataset underwent preprocessing step before training process to prepare raw data for the input of machine learning model. In this preprocess step, first, the arbitration ID and data fields in the dataset which includes hexadecimal values were converted to decimal values, ensuring the feed to the machine learning model with numeric values. In addition, class labels are stored in the dataset with alphabetical values. These values were converted by using Label Encoder approach. In this approach, categorical columns are converted into numerical ones. Assume a dataset with a class column having values Normal, Fuzzy, Replay, and Spoofing. Label Encoding transform these alphabetical values into numerical values as 0, 1, 2, and 3, respectively.

Secondly, a ECU transmits data changing 1 to 8 bytes. If it transmits 5 bytes, the remaining 3 bytes in the dataset are None values. These None values are filled with zeros for the entire dataset due to that machine learning models cannot feed with None values as well.

Thirdly, timestamp column in the dataset is in Unix format and is represented as a single value which includes year, month, day, hour, minutes, second, and milisecond information. Therefore, this timestamp column were separated into seven columns as year, month, day, hour, minutes, second, and milisecond. Then, year, month, day, and hour columns were dropped due to that they contain the same value along whole dataset. Thus, while the model speed increases with less data size, it will have no effect in the accuracy of the model.

Another step before training process is feature extraction. This step is done by creating new features from existing ones to increase the accuracy of the machine learning model. In this study, two new features were exracted. One of these feature is Time Interval. It is created by using existing timestamp feature. This derived feature represents the time difference between consecutive messages. By calculating the interval between successive timestamps, the feature helps capture patterns related to the timing and frequency of messages. This feature in CAN bus data can be useful in detecting specific types of attacks, particularly those that disrupt the normal timing patterns of message transmission.

Table 2
An Overview of the CAN Bus Datasets Utilized in MetaCAN.

Name	Vehicle	Attack Type	Number of Messages
Survival Analysis (Sonata) [76]	Hyundai YF Sonata	Flooding	149,547
		Fuzzy	135,670
		Malfunction	132,651
		Normal	117,173
Survival Analysis (Soul) [76]	Kia Soul	Flooding	181,901
		Fuzzy	249,990
		Malfunction	173,436
		Normal	192,516
Survival Analysis (Spark) [76]	Chevrolet Spark	Flooding	120,570
		Fuzzy	65,665
		Malfunction	79,787
		Normal	136,934
Car Hacking: Attack & Defense Challenge 2020 (Preliminary Training) [77,78]	Hyundai Avante CN7	Flooding	154,746
		Spoofing	7,758
		Replay	47,550
		Fuzzy	90,940
		Normal	3,449,260
Car Hacking: Attack & Defense Challenge 2020 (Preliminary Submission) [77,78]	Hyundai Avante CN7	Flooding	191,679
		Spoofing	42,583
		Replay	62,881
		Fuzzy	96,693
		Normal	3,358,210
Car Hacking: Attack & Defense Challenge 2020 (Final Submission) [77,78]	Hyundai Avante CN7	Spoofing	149,999
		Fuzzy	29,999
		Normal	1,090,312
OTIDS [75]	Kia Soul	DoS	656,579
		Fuzzy	591,990
		Impersonation	995,472
		Normal	2,369,868

Table 3
A Few Examples of the CAN Bus Data.

Timestamp	CAN ID	DLC	Data	Class
37.022496	043f	8	00 40 60 ff 77 80 09 00	Fuzzy
119.228405	05a2	4	20 00 a5 01	DoS
154.347077	0220	8	f0 03 01 04 0c 00 3d 10	Normal
226.884105	0220	8	44 b4 b4 ba 17 3e cc fe	Fuzzy
316.142643	0165	8	13 d8 7f 00 00 00 0d b9	Normal
316.142828	02b0	5	1d 00 00 07 6d	Normal
1037.589628	0165	8	11 e8 7f 00 00 00 0c 8a	Normal
1481193380.717519	0329	8	85 b3 80 8c 11 2b 00 10	Impersonation

Under normal driving conditions, the time intervals between recurring messages, especially those with the same CAN ID, tend to follow a stable and predictable pattern. However, in the presence of certain attacks, such as DoS or flooding, this pattern is often disrupted due to the injection of messages at abnormally high frequencies. By highlighting these irregularities, the Time Interval feature becomes useful in identifying anomalies in the communication flow that would otherwise be difficult to detect through static features alone. To highlight the benefit of the newly created Time Interval feature, the first 140,000 data samples from the Car Hacking: Attack & Defense Challenge 2020 dataset are visualized in Fig. 8. The visualization clearly demonstrates that the Time Interval aligns closely with DoS attacks. Specifically, during a DoS attack, the Time Interval decreases significantly, aiding the machine learning model in improving its learning and detection capabilities.

The other created feature is the ID Repetition Count. This feature calculates the number of times a specific message ID is repeated within a predefined time window. In this study, this time window is determined as 100 milliseconds.

By counting the repetitions of message IDs, this feature helps to identify irregular patterns in message transmission. For example, in replay attacks, an unusually high repetition count within the time window may indicate that a legitimate message is being replayed multiple times by an attacker. Similarly, in DoS attacks, a flood of repeated high-priority messages can lead to a sharp increase in the repetition count for certain IDs. By tracking the frequency of each message ID, this feature captures abnormal surges in message transmission that may indicate malicious activity. Under normal driving conditions, the repetition of CAN IDs generally follows a predictable pattern based on system requirements and sensor update rates. However, during attacks such as replay, spoofing, or DoS, this pattern is disrupted. For instance, in replay attacks, a legitimate message is often resent multiple times by an attacker within a short duration, leading to an unusually high repetition count for that particular ID. Similarly, in DoS attacks, the network is overwhelmed with a flood of repeated high-priority messages, causing a sharp increase in the repetition count of specific IDs. By identifying such irregularities, this feature serves as a valuable indicator of anomalous behavior in the CAN bus. To show the advantage of the created ID Repetition Count feature,

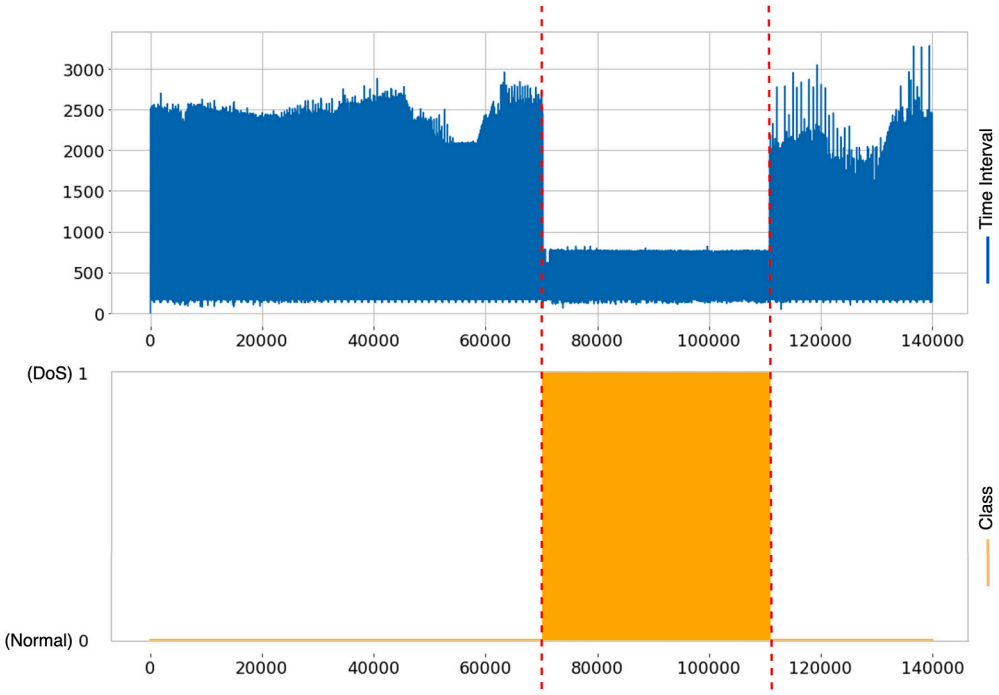


Fig. 8. Representation of the Created Time Interval Feature.

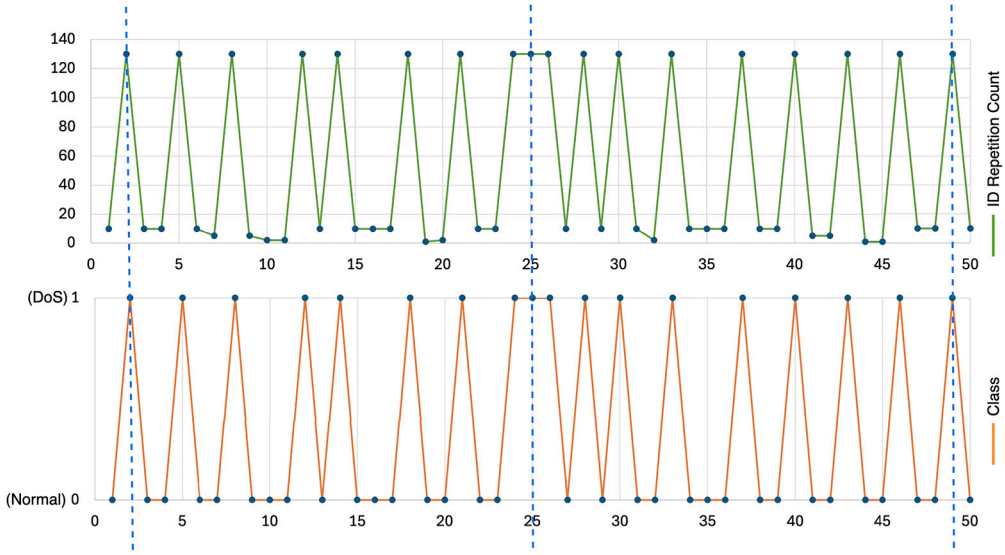


Fig. 9. Representation of the Created ID Repetition Count Feature.

50 data samples (between 99995 and 100045) from Car Hacking: Attack & Defense Challenge 2020 dataset are visualized in Fig. 9. As can be seen clearly, the ID Repetition Count is very compatible with DoS attack. It means that when DoS attack is performed, ID repetition count increases dramatically, which helps to the machine learning model learns better.

These created features provide an additional layer of insight into message transmission behavior. Thereby, it improves the detection capabilities of IDSs for CAN bus networks.

6. Evaluation results and discussion

In MetaCAN, a hybrid optimization approach combining PSO and CS algorithms was employed to optimize the hyperparameters of the XGBoost classifier. The parameters for each optimization algorithm were carefully selected to balance between exploration and exploitation during the search process. For the PSO algorithm, a swarm size of 10 particles was used with an inertia weight (w) of 0.7 to control the balance between exploration and exploitation. The acceleration coefficients c_1 and c_2 were both set to 1.7 to maintain a steady influence of individual and collective experiences. The number of iterations was set to 25 to ensure convergence within a reasonable time frame. For the CS algorithm, the population size was also set to 10 with 25 iterations. The discovery rate (P_a) which controls the probability of abandoning worse solutions was set to 0.25. Additionally, Lévy distribution parameter (L) was set to 1.5 and the step size (s) for Lévy flights was defined as 0.01 to facilitate the generation of new solutions based on random walks.

The hyperparameters of XGBoost, and their upper and lower bounds are given in Table 4. The table also includes the optimal values obtained through the optimization process for each dataset. These optimal values represent the best-performing configurations found during the optimization process, which directly impact the overall performance of the proposed MetaCAN IDS. The variations in the optimal hyperparam-

Table 4
The Hyperparameters of XGBoost, their Upper and Lower Bounds, and Optimal Values Obtained through the Optimization Process.

Name	Upper and Lower Bounds	Survival Analysis (Sonata)	Survival Analysis (Soul)	Survival Analysis (Spark)	Car Hacking: Attack & Defense Challenge 2020 (Preliminary Training)	Car Hacking: Attack & Defense Challenge 2020 (Preliminary Submission)	Car Hacking: Attack & Defense Challenge 2020 (Final Submission)	OTIDS
n_estimators	[10, 200]	61	106	144	200	188	104	170
min_child_weight	[1, 10]	8	1	1	5	6	1	5
eta	[0.05, 0.3]	0.292820	0.3	0.3	0.267641	0.253752	0.098374	0.262742
alpha	[0, 0.0001]	3.1e-5	9.7e-5	5.2e-5	8.8e-5	7.3e-5	7.9e-05	1.0e-04
lambda	[0.5, 1]	0.633056	1.0	0.5	0.734385	1.0	0.910664	1.0
subsample	[0.5, 1]	0.551216	0.5	0.883618	0.815631	1.0	0.547365	0.840473
max_depth	[3, 10]	5	3	4	10	10	9	10

eter values across different datasets highlight the adaptability of the optimization approach to distinct data characteristics and attack scenarios, ensuring robust intrusion detection performance across diverse operating conditions.

6.1. Evaluation metrics

The performance results of the proposed IDS were evaluated by using widely used classification metrics such as accuracy, precision, recall, and F1-score. These metrics are calculated by using TP (True Positive), TN (True Negative), FP (False Positive), and FN (False Negative) values as follows [79]:

$$Accuracy = \frac{(TP + TN)}{(TP + TN + FP + FN)} \tag{13}$$

$$Precision = \frac{TP}{(TP + FP)} \tag{14}$$

$$Recall = \frac{TP}{(TP + FN)} \tag{15}$$

$$F1 - Score = 2 \cdot (\frac{Precision \cdot Recall)}{(Precision + Recall)} \tag{16}$$

6.2. Performance analysis of the MetaCAN

Fig. 10(a, b, & c) illustrate the convergence graphs of the optimization process of the proposed MetaCAN IDS for the Survival Analysis (Sonata), Survival Analysis (Soul), and Survival Analysis (Spark) datasets, respectively. The first 25 iterations correspond to the PSO algorithm, while the remaining 25 iterations correspond to the CS algorithm. As can be seen clearly, during the first 25 iterations, first metaheuristic which is PSO struggles to find the optimal solution and gets trapped in local optima. In contrast, the second metaheuristic which is CS effectively reaches the optimal solution thanks to operating in a narrowed search space. Notably, in Fig. 10(a&b), accuracies decrease slightly at the beginning of the second metaheuristic. This behavior is expected as the proposed MetaCAN IDS updates the lower and upper bounds of the XGBoost hyperparameters by $\pm 10\%$ of the values found by the first metaheuristic. In other words, a solution found by the PSO algorithm in the broader search space may outperform any solution in the newly constrained search space at the start of the CS phase.

Fig. 11(a, b, & c) represent the convergence graphs of the optimization process of the proposed MetaCAN IDS for the Car Hacking: Attack & Defense Challenge 2020 (Preliminary Training), Car Hacking: Attack & Defense Challenge 2020 (Preliminary Submission), and Car Hacking: Attack & Defense Challenge 2020 (Final Submission) datasets, respectively. On the contrary to Survival Analysis dataset, first metaheuristic which is PSO reaches the optimal solution and second metaheuristic which is CS needs more iterations to reach it. In this scenario, the MetaCAN chooses the optimal solution found by the first metaheuristic.

Fig. 12a illustrates the convergence graph of the optimization process of proposed MetaCAN IDS for the OTIDS dataset. Similar to Car Hacking: Attack & Defense Challenge 2020 dataset, the first metaheuristic reaches the optimal solution and the MetaCAN chooses the optimal solution found by the first metaheuristic.

Fig. 10(d, e, & f) illustrate the confusion matrixes of the optimal solutions of proposed MetaCAN IDS for the Survival Analysis (Sonata), Survival Analysis (Soul), and Survival Analysis (Spark) datasets, respectively. The performance results of MetaCAN are given in Table 5 using the metrics such as accuracy, precision, recall, and F1-score. This table also presents a performance comparison between MetaCAN and existing IDSs for the Survival Analysis dataset. The proposed MetaCAN outperforms all existing IDSs across all attack types.

For the Survival Analysis (Sonata) dataset, MetaCAN achieves 1.0 accuracy, precision, recall, and F1-score for flooding and malfunction attacks. It outperforms other IDSs which generally struggle with these types of attacks. For example, MetaCAN has achieved 2.3% and 4.4% higher accuracies than [43] and [52] for flooding attack, respectively. Similarly, in the Survival Analysis (Soul) dataset, MetaCAN maintains near-perfect performance across all attack types, while other IDSs exhibit noticeable drops in recall and F1-score for specific attack types. For example, some IDSs show recall or F1-score values below 0.95 for fuzzy and malfunction attacks. This indicates difficulty in accurately identifying these types of anomalies. For example, MetaCAN has achieved 11.1% higher recall and 5.6% higher F1-score than [43] for flooding attack. It has also achieved 1.9% higher accuracy and 4.2% higher precision than [49] for malfunction attack. Furthermore, the Survival Analysis (Spark) dataset further reinforces robustness of MetaCAN. It achieves 1.0 accuracy, precision, recall, and F1-score for flooding and malfunction attacks, while competing IDSs show a decline in performance, particularly for the fuzzy attack type. Notably, some IDSs like in [52] achieve only around 0.8 accuracy for all attack types, whereas MetaCAN reaches nearly 1.0 across all metrics, representing approximately 25% higher accuracy.

The comparison highlights that MetaCAN consistently achieves superior performance across all datasets and attack types. Specifically, MetaCAN demonstrates near-perfect performance in detecting all attack types, with accuracy, precision, recall, and F1-score values frequently reaching 1.0. In contrast, existing IDSs show varying levels of effectiveness depending on the dataset and attack type.

Fig. 11(d, e, & f) represent the confusion matrixes of the optimal solutions of the proposed MetaCAN IDS for the Car Hacking: Attack & Defense Challenge 2020 (Preliminary Training), Car Hacking: Attack & Defense Challenge 2020 (Preliminary Submission), Car Hacking: Attack & Defense Challenge 2020 (Final Submission) datasets, respectively. The performance results of the MetaCAN are given in Table 6 using the metrics such as accuracy, precision, recall, and F1-score. The performance of the proposed MetaCAN is remarkable across all attack types and datasets from the Car Hacking: Attack & Defense Challenge 2020. MetaCAN achieves near-perfect in all metrics for detecting various attack types such as flooding, spoofing, replay, and fuzzy attacks. Notably, MetaCAN consistently reaches a perfect score of 1.0 for flooding attack across all datasets, demonstrating its robustness in identifying this critical threat.

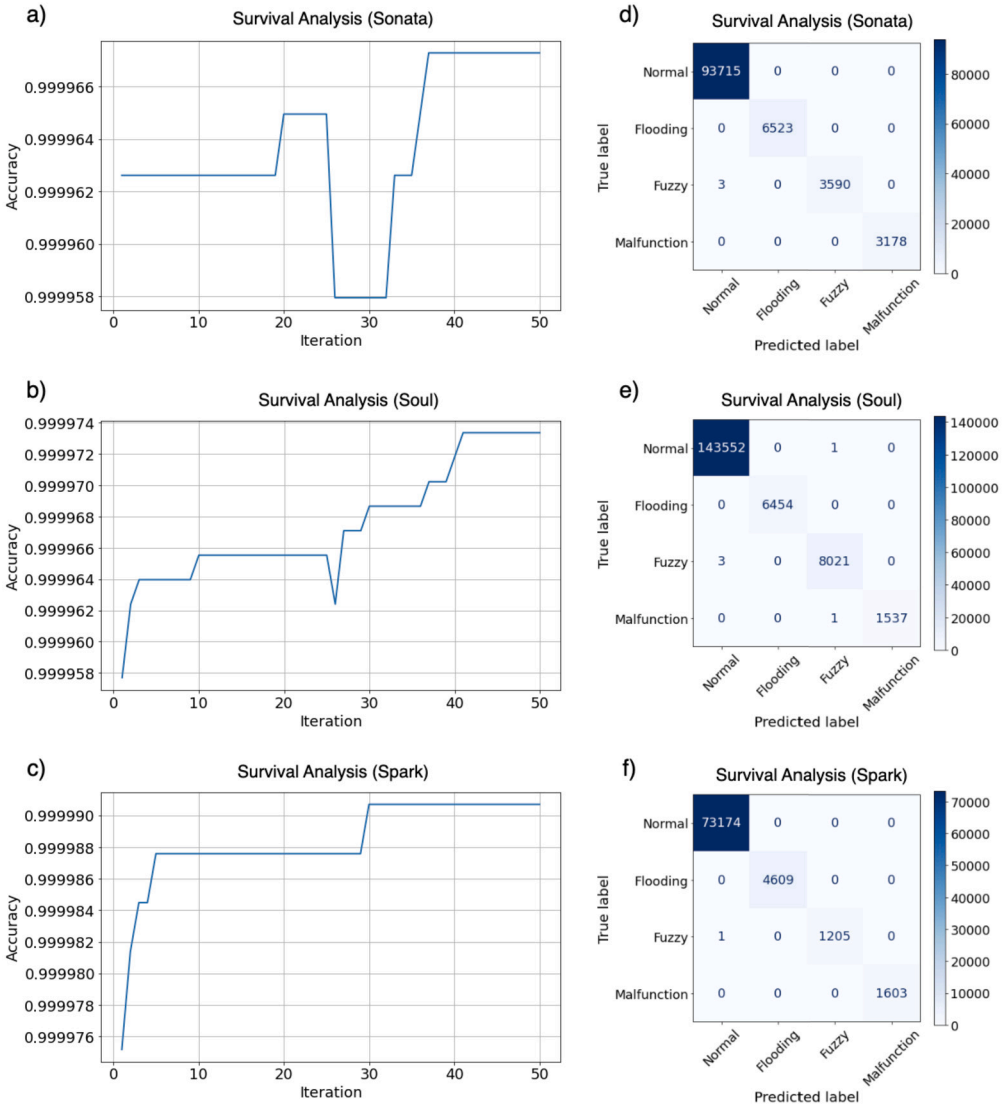


Fig. 10. The Convergence Graphs (a, b, and c) and the Confusion Matrixes (d, e, and f) for the Survival Analysis (Sonata), Survival Analysis (Soul), and Survival Analysis (Spark) Datasets, Respectively.

Furthermore, Table 7 presents the performance comparison between MetaCAN and existing IDSs for the Car Hacking: Attack & Defense Challenge 2020 dataset. The existing IDSs shared their performance results for binary classification. Therefore, the binary classification performance of MetaCAN has also been evaluated for the comparison. MetaCAN achieves the highest accuracy of 0.999984 and a perfect precision of 1.0, recall of 0.999889, and F1-score of 0.999945. In contrast, the other IDSs show varied and generally lower performance, with accuracy values ranging from 0.9514 to 0.9952. For example, MetaCAN has achieved 5.1% higher accuracy than [81]. Hence, MetaCAN outperforms all other IDSs and proves to be the most reliable IDS solution for CAN bus security in this comparison.

Fig. 12b illustrates the confusion matrix of the optimal solution of proposed MetaCAN IDS for the OTIDS dataset. The performance result of MetaCAN is given in Table 8 using the metrics such as accuracy, precision, recall, and F1-score. This table also presents a performance comparison between MetaCAN and existing IDSs for the OTIDS dataset. Considering the accuracy metric, MetaCAN outperforms all existing IDSs across all attack types. However, when evaluating other metrics, one existing IDS which is represented in [84] shows slightly better performance for the DoS and Fuzzy attack types. Nonetheless, MetaCAN achieves the best performance for the impersonation attack type. For example, MetaCAN has achieved 12.8% and 4.2% higher accuracies than [85] and [86] for impersonation attack, respectively. Similarly, for fuzzy attack, it has also achieved 2.8% and 13.7% higher accuracies than [85] and [86], respectively.

Fig. 13 demonstrates overall accuracy results of proposed MetaCAN IDS across multiple datasets, with particularly strong results in both Survival Analysis and Car Hacking: Attack & Defense Challenge 2020 datasets. The model achieved remarkably high accuracy scores exceeding 0.999 for all three Survival Analysis subsets (Sonata: 0.999972, Soul: 0.999969, and Spark: 0.999988), indicating nearly perfect detection capability. In the Car Hacking Attack & Defense Challenge 2020 dataset, the model maintained consistent performance across different subsets of it, with accuracies of 0.999224 for Preliminary Training, 0.998130 for Preliminary Submission, and 0.999992 for Final Submission. Lastly, the model yielded a slightly lower but still impressive accuracy of 0.984237 on OTIDS dataset. This result is notable given the typically challenging nature of intrusion detection tasks.

In summary, the comprehensive evaluation across multiple datasets and attack types demonstrates the superior performance and robustness of MetaCAN compared to existing IDSs in the literature. MetaCAN consistently achieves high performance across a wide range of attacks. Un-

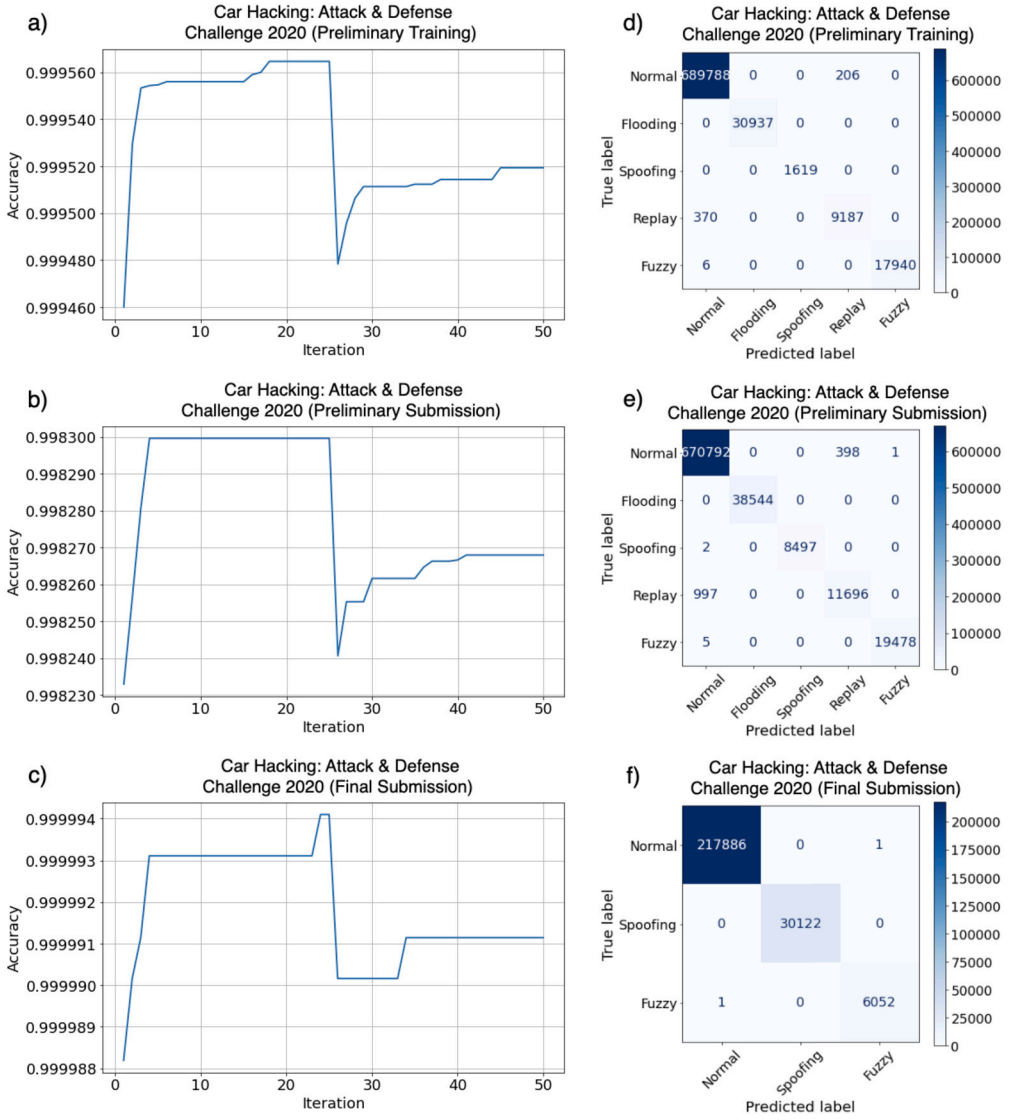


Fig. 11. The Convergence Graphs (a, b, and c) and the Confusion Matrixes (d, e, and f) for the Car Hacking: Attack & Defense Challenge 2020 (Preliminary Training), Car Hacking: Attack & Defense Challenge 2020 (Preliminary Submission), and Car Hacking: Attack & Defense Challenge 2020 (Final Submission) Datasets, Respectively.

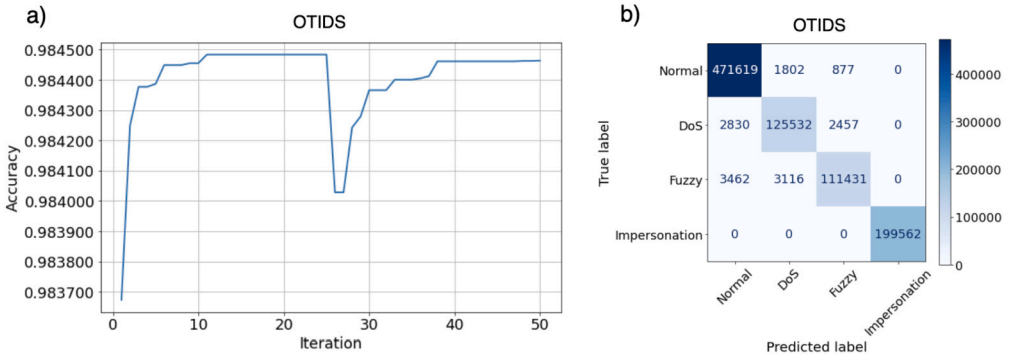


Fig. 12. The Convergence Graph (a) and the Confusion Matrix (b) for the OTIDS Dataset.

like other IDSs that typically perform well for only a few attack types or specific datasets, MetaCAN offers a broader multi-class detection ability that works well across various scenarios. Its ability to maintain consistent performance across different real-world datasets highlights its practical applicability for securing vehicular communication systems. These results validate the adaptability and reliability of the proposed hybrid metaheuristic optimization approach, making MetaCAN a highly effective and scalable solution for CAN bus security.

Table 5
The Performance of the MetaCAN and the Comparison of it with the Existing IDSs for the Survival Analysis Dataset.

Dataset	Reference	Attack Type	Accuracy	Precision	Recall	F1-Score
Survival Analysis (Sonata)	[40]	Normal	-	-	**1.0**	**1.0**
		Flooding	-	-	**1.0**	**1.0**
		Fuzzy	-	-	0.9995	0.9996
		Malfunction	-	-	**1.0**	**1.0**
	[43]	Normal	-	-	-	-
		Flooding	0.9783	**1.0**	0.9000	0.9474
		Fuzzy	0.9998	0.9998	0.9908	0.9998
		Malfunction	0.9999	0.9992	**1.0**	0.9992
	[49]	Normal	-	-	-	-
		Flooding	0.9973	0.9972	0.9972	0.9272
		Fuzzy	**1.0**	**1.0**	**1.0**	**1.0**
		Malfunction	0.9985	**1.0**	0.9964	0.9982
	[52]	Normal	-	-	-	-
		Flooding	0.95790	0.98921	0.04790	0.96434
		Fuzzy	0.99583	0.99584	0.99583	0.99583
		Malfunction	0.96598	0.98490	0.96598	0.97223
	MetaCAN	Normal	**0.999972**	**0.999968**	**1.0**	**0.999984**
		Flooding	**1.0**	**1.0**	**1.0**	**1.0**
		Fuzzy	**0.999972**	**1.0**	0.999165	0.999582
		Malfunction	**1.0**	**1.0**	**1.0**	**1.0**
Survival Analysis (Soul)	[40]	Normal	-	-	**1.0**	0.9994
		Flooding	-	-	**1.0**	**1.0**
		Fuzzy	-	-	0.9469	0.9701
		Malfunction	-	-	0.9697	0.9087
	[43]	Normal	-	-	-	-
		Flooding	0.9818	**1.0**	0.9000	0.9474
		Fuzzy	0.9993	0.9999	0.9958	0.9978
		Malfunction	0.9971	**1.0**	0.9322	0.9649
	[49]	Normal	-	-	-	-
		Flooding	0.9857	0.9692	0.9974	0.9831
		Fuzzy	0.9992	**1.0**	0.9984	0.9992
		Malfunction	0.9815	0.9592	0.9903	0.9745
	[80]	Normal	-	0.998	0.9968	0.9974
		Flooding	-	**1.0**	**1.0**	**1.0**
		Fuzzy	-	0.9684	0.9784	0.9733
		Malfunction	-	0.9965	0.9995	0.9980
	MetaCAN	Normal	**0.999975**	**0.999979**	**0.999993**	**0.999986**
		Flooding	**1.0**	**1.0**	**1.0**	**1.0**
		Fuzzy	**0.999969**	0.999751	**0.999626**	**0.999688**
		Malfunction	**0.999994**	**1.0**	**0.999350**	**0.999675**
Survival Analysis (Spark)	[40]	Normal	-	-	0.9998	0.9997
		Flooding	-	-	**1.0**	**1.0**
		Fuzzy	-	-	0.9823	0.9831
		Malfunction	-	-	**1.0**	**1.0**

(*continued on next page*)

Table 5 (*continued*)

Dataset	Reference	Attack Type	Accuracy	Precision	Recall	F1-Score
	[43]	Normal	-	-	-	-
		Flooding	0.9813	**1.0**	0.9000	0.9474
		Fuzzy	0.9907	0.9734	0.9200	0.9460
		Malfunction	0.9998	0.9980	**1.0**	0.9990
	[49]	Normal	-	-	-	-
		Flooding	0.9983	0.9965	**1.0**	0.9983
		Fuzzy	0.9970	0.9892	**1.0**	0.9946
		Malfunction	0.9975	0.9936	**1.0**	0.9968
	[52]	Normal	-	-	-	-
		Flooding	0.73693	0.99436	0.73693	0.84289
		Fuzzy	0.81890	0.89196	0.81890	0.82917
		Malfunction	0.76992	0.99474	0.76992	0.86477
	[80]	Normal	-	0.9983	0.9995	0.9989
		Flooding	-	**1.0**	**1.0**	**1.0**
		Fuzzy	-	0.9885	0.9939	0.9912
		Malfunction	-	0.9917	0.9978	0.9947
	MetaCAN	Normal	**0.999988**	**0.999986**	**1.0**	**0.999993**
		Flooding	**1.0**	**1.0**	**1.0**	**1.0**
		Fuzzy	**0.999988**	**1.0**	0.999171	**0.999585**
		Malfunction	**1.0**	**1.0**	**1.0**	**1.0**

Table 6
MetaCAN Performance when Applied to the Car Hacking: Attack & Defense Challenge 2020 Dataset.

Dataset	Attack Type	Accuracy	Precision	Recall	F1-Score
Car Hacking: Attack & Defense Challenge 2020 (Preliminary Training)	Normal	0.999224	0.999455	0.999701	0.999578
	Flooding	1.0	1.0	1.0	1.0
	Spoofing	1.0	1.0	1.0	1.0
	Replay	0.999232	0.978069	0.961285	0.969604
	Fuzzy	0.999992	1.0	0.999666	0.999833
Car Hacking: Attack & Defense Challenge 2020 (Preliminary Submission)	Normal	0.998130	0.998505	0.999406	0.998955
	Flooding	1.0	1.0	1.0	1.0
	Spoofing	0.999997	1.0	0.999765	0.999882
	Replay	0.998141	0.967091	0.921453	0.943720
	Fuzzy	0.999992	0.999949	0.999743	0.999846
Car Hacking: Attack & Defense Challenge 2020 (Final Submission)	Normal	0.999992	0.999995	0.999995	0.999995
	Spoofing	1.0	1.0	1.0	1.0
	Fuzzy	0.999992	0.999835	0.999835	0.999835

Table 7
Effectiveness of MetaCAN Compared to a Selection of Existing IDSs when Applied to the Car Hacking: Attack & Defense Challenge 2020 Dataset.

Reference	Accuracy	Precision	Recall	F1-Score
[49]	0.9817	0.9827	0.9645	0.9818
[45]	0.9952	0.9977	0.9954	0.9965
[42]	0.9810	0.9814	0.9804	0.9542
[44]	0.9847	0.9804	0.9891	0.9846
[81]	0.9514	0.9447	0.9278	0.9631
[82]	0.9901	0.9899	0.9824	0.9883
[83]	0.9991	0.9991	0.9991	0.9991
MetaCAN	**0.999984**	**1.0**	**0.999889**	**0.999945**

6.3. Analysis of attack-based detection results

To evaluate the robustness of MetaCAN against different attack types, we conducted a detailed performance analysis across five major attack categories: DoS, fuzzy, replay, spoofing, and malfunction. Fig. 14 presents the comparative results in terms of accuracy, precision, recall, and F1-score. These results were obtained by averaging the accuracy, precision, recall, and F1-score values for each attack type across multiple datasets, ensuring a comprehensive and dataset-independent evaluation.

In terms of accuracy (Fig. 14a), all attack types were detected with very high performance, with values exceeding 0.998. Spoofing and malfunction attacks achieved near-perfect detection with accuracies of 0.999999 and 0.999998, respectively. These results suggest

Table 8
The Performance of the MetaCAN and the Comparison of it with the Existing IDSs for the OTIDS Dataset.

Reference	Attack Type	Accuracy	Precision	Recall	F1-Score
[41]	Normal	-	0.9369	0.9391	0.9379
	DoS	-	0.9389	0.9422	0.9406
	Fuzzy	-	0.9488	0.9531	0.9509
	Impersonation	-	0.9377	0.9425	0.9401
[51]	Normal	-	0.92	0.99	0.95
	DoS	-	0.89	0.78	0.83
	Fuzzy	-	0.92	0.74	0.82
	Impersonation	-	**1.0**	**1.0**	**1.0**
[87]	Normal	-	**0.9996**	**0.9985**	**0.9990**
	DoS	-	**1.0**	**0.9994**	**0.9997**
	Fuzzy	-	**1.0**	**1.0**	**1.0**
	Impersonation	-	0.9980	0.9996	0.9988
[84]	Normal	-	0.8007	0.9932	0.8866
	DoS	-	0.8703	0.7214	0.7888
	Fuzzy	-	0.9109	0.7148	0.8010
	Impersonation	-	0.9206	0.6156	0.7378
[85]	Normal	0.8325	0.8013	0.8857	0.8414
	DoS	0.8871	0.5864	0.7962	0.6754
	Fuzzy	0.9626	0.9277	0.7764	0.8453
	Impersonation	0.8862	0.8901	0.5483	0.6786
[86]	Normal	0.91	0.9	0.91	0.9
	DoS	0.98	0.97	0.98	0.98
	Fuzzy	0.87	0.94	0.88	0.91
	Impersonation	0.96	**1.0**	0.96	0.98
MetaCAN	Normal	**0.990277**	0.986834	0.986834	0.990579
	DoS	**0.988940**	0.962300	0.959585	0.960941
	Fuzzy	**0.989257**	0.970949	0.944258	0.957418
	Impersonation	**1.0**	**1.0**	**1.0**	**1.0**

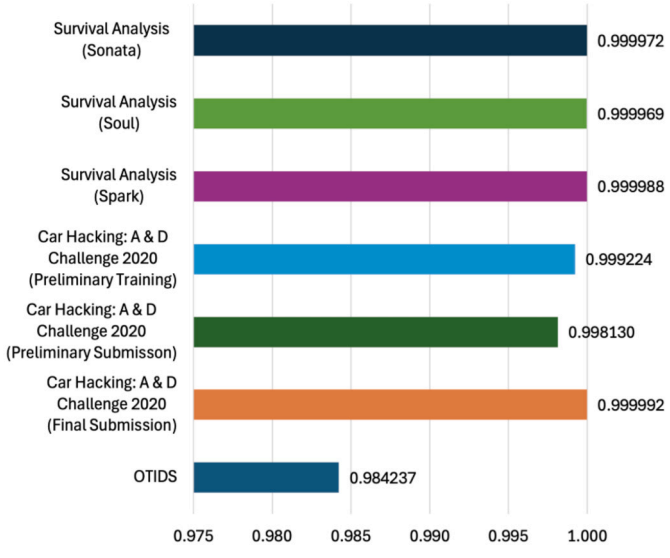


Fig. 13. Overall Accuracies of MetaCAN Across all Three Datasets: Survival Analysis, Car Hacking: Attack & Defense Challenge 2020, and OTIDS.

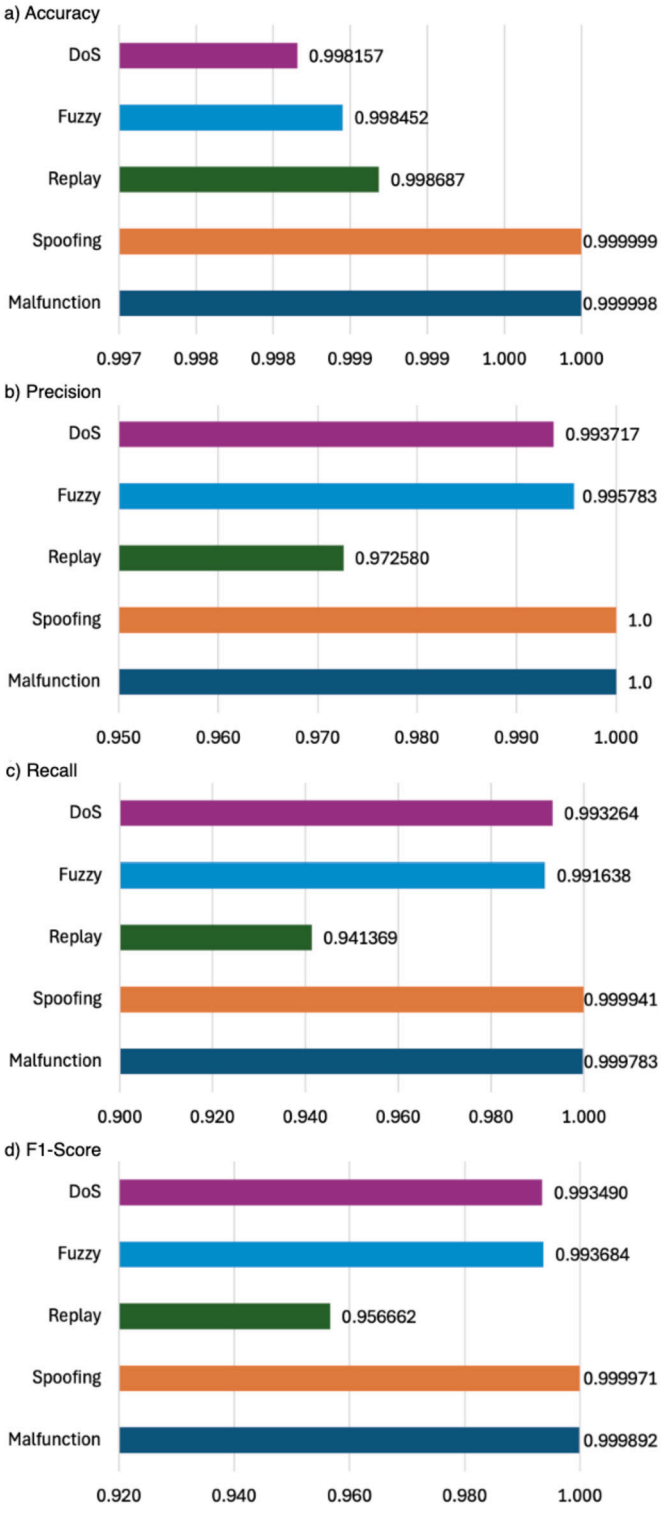


Fig. 14. The Achieved Detection Performance of MetaCAN Across Various Attack Types such as DoS, Fuzzy, Replay, Spoofing, and Malfunction.

that the model is highly effective in distinguishing these attack types from normal behavior. Replay attacks, which typically involve the retransmission of previously captured packets, recorded an accuracy of 0.998687, indicating the model's robustness even against temporal manipulation. Fuzzy and DoS attacks were also accurately identified, with accuracies of 0.998452 and 0.998157, respectively.

The precision results (Fig. 14b) further validate the model's ability to avoid false positives. Notably, spoofing and malfunction attacks

Table 9
Timing Analysis of MetaCAN (in seconds), including both Training and Testing Durations.

Dataset	PSO	CS	Total Training Time	Testing Time
Survival Analysis (Sonata)	3293.46	12086.35	15379.81	0.037465
Survival Analysis (Soul)	5191.28	15450.09	20641.36	0.088082
Survival Analysis (Spark)	2278.01	9834.90	12112.91	0.033502
Car Hacking: Attack & Defense Challenge 2020 (Preliminary Training)	24356.48	54328.97	78685.44	0.555917
Car Hacking: Attack & Defense Challenge 2020 (Preliminary Submission)	57542.50	157209.62	214752.12	1.489847
Car Hacking: Attack & Defense Challenge 2020 (Final Submission)	7112.89	20634.04	27746.93	0.143479
OTIDS	83566.84	254947.40	338514.24	2.406797
Mean Value	26191.64	74927.34	101118.97	0.679298

achieved a precision score of 1.0, confirming zero false alarms for these categories. The lowest precision, recorded for replay attacks (0.972580), may reflect the challenge in differentiating replayed signals from legitimate traffic due to their high similarity.

Recall results (Fig. 14c) reveal how effectively MetaCAN captures true positive attack instances. Spoofing and malfunction again led with recall scores of 0.999941 and 0.999783, respectively, while replay exhibited the lowest recall (0.941369). This indicates that although replay attacks were mostly detected, a small portion was missed, pointing to a potential area for enhancement.

Lastly, the F1-score (Fig. 14d), which balances precision and recall, affirms the consistent high performance of MetaCAN across most attack types. Replay again scored lower at 0.956662, reflecting the impact of its relatively lower recall. In contrast, the other four attack types recorded F1-scores above 0.993, highlighting the model's strong and balanced detection capability.

These results demonstrate that MetaCAN provides reliable detection performance across a wide spectrum of cyberattacks, maintaining minimal variance across metrics. This consistency underscores the potential of the model for real-world deployment in CAN-based vehicular networks.

6.4. Timing analysis of MetaCAN

To assess the computational efficiency of the MetaCAN framework, a detailed timing analysis was conducted across all datasets used in this study. Table 9 presents a breakdown of the total training time, highlighting the durations for PSO, CS, and testing phases separately. For better visualization, Fig. 15 graphically represents these time measurements. MetaCAN employs a hybrid metaheuristic optimization strategy by combining PSO and CS to tune the hyperparameters of the XGBoost classifier. The analysis shows that PSO consistently converges faster than CS across all datasets, contributing to improved training efficiency when used together.

On average, PSO (26191.64 seconds) is approximately 65% faster than CS (74927.34 seconds). This performance gap underscores the quicker convergence of PSO, which helps reduce overall training time when compared to using CS alone. Therefore, the hybrid approach benefits from the speed of PSO while retaining the exploratory capability of CS. Despite the metaheuristic tuning process being computationally intensive, particularly for large datasets like OTIDS and the Car Hacking: Attack & Defense Challenge 2020, the testing times remain consistently low. This hybrid configuration effectively balances the exploration and exploitation aspects of the optimization process. Although training time is relatively high due to the iterative nature of metaheuristics, it is a one-time cost. Once trained, the model operates based on the reported test time. In conclusion, this timing analysis demonstrates the advantages of using the hybrid approach and highlights the practical feasibility of MetaCAN in intrusion detection scenarios, where model training can be performed offline and inference remains swift and efficient.

Comparing the runtime performance of MetaCAN with alternative IDS solutions requires that all models be evaluated under the same hardware and environmental conditions to ensure fairness. However, most researchers report runtime results based on their own specific hardware setups, which makes direct comparison difficult. Furthermore, as clearly shown in Table 9, the execution times of MetaCAN vary depending on the dataset used. Therefore, runtime comparisons should ideally be conducted using the same datasets as well. Nonetheless, presenting available benchmarks remains valuable and provides useful insight into the computational performance of MetaCAN.

For the Survival Analysis dataset, the IDS named G-IDCS [49] achieved an average detection time of 0.7292 seconds. In contrast, MetaCAN demonstrated a significantly lower average testing time of 0.053 seconds, outperforming G-IDCS. In another study [52], the reported detection time was approximately 0.035 seconds. Although MetaCAN's average testing time of 0.053 seconds is slightly higher, it remains competitive.

For the Car Hacking: Attack & Defense Challenge 2020 dataset, a random forest-based IDS [81] reported an average detection time of 0.2 seconds. The average testing time of MetaCAN for the same dataset is 0.72 seconds. However, it is important to highlight that the study in [81] evaluated binary classification, whereas MetaCAN was tested for multi-class classification, which inherently demands more computational resources.

Regarding the OTIDS dataset, DivaCAN [51] recorded a testing time of 3 seconds, whereas MetaCAN achieved a lower average testing time of 2.41 seconds, indicating better efficiency. Similarly, the IDS presented in [86] had a detection time of approximately 7 seconds, which MetaCAN outperformed significantly with its 2.41 seconds testing time.

However, it is obvious that the training time of MetaCAN is higher compared to its alternative IDSs because it relies on metaheuristic optimization, which is computationally expensive. While MetaCAN incurs higher training costs due to the hybrid PSO-CS optimization process, its testing time remains consistently low (average of 0.679298 seconds across all datasets), which is suitable for near real-time or real-time detection tasks in vehicular systems.

It is important to emphasize that the training process is performed offline, allowing the computational overhead of metaheuristic tuning to be excluded from the real-time deployment phase. Once trained, the MetaCAN model operates efficiently, making it viable for on-vehicle deployment. These additions collectively highlight the practical applicability and effectiveness of MetaCAN even under hardware-constrained scenarios.

7. Conclusions and future work

The CAN bus is highly vulnerable to cyberattacks due to its simple architecture, which lacks crucial security features such as authentication, proper access control, and encryption. To address these vulnerabilities and ensure passenger safety, the implementation of an intrusion detection system (IDS) becomes imperative. In this study, MetaCAN which is a machine learning based IDS optimized through an adaptive hybrid metaheuristic approach is proposed to defend the CAN bus against cyberattacks. This hybrid method combines the PSO and CS algorithms by leveraging their unique strengths such as the fast convergence of PSO and the powerful global search capability of CS. Furthermore, the accuracy of MetaCAN is enhanced through a feature engineering process by introducing two new features (see Fig. 8 and 9). MetaCAN demonstrates

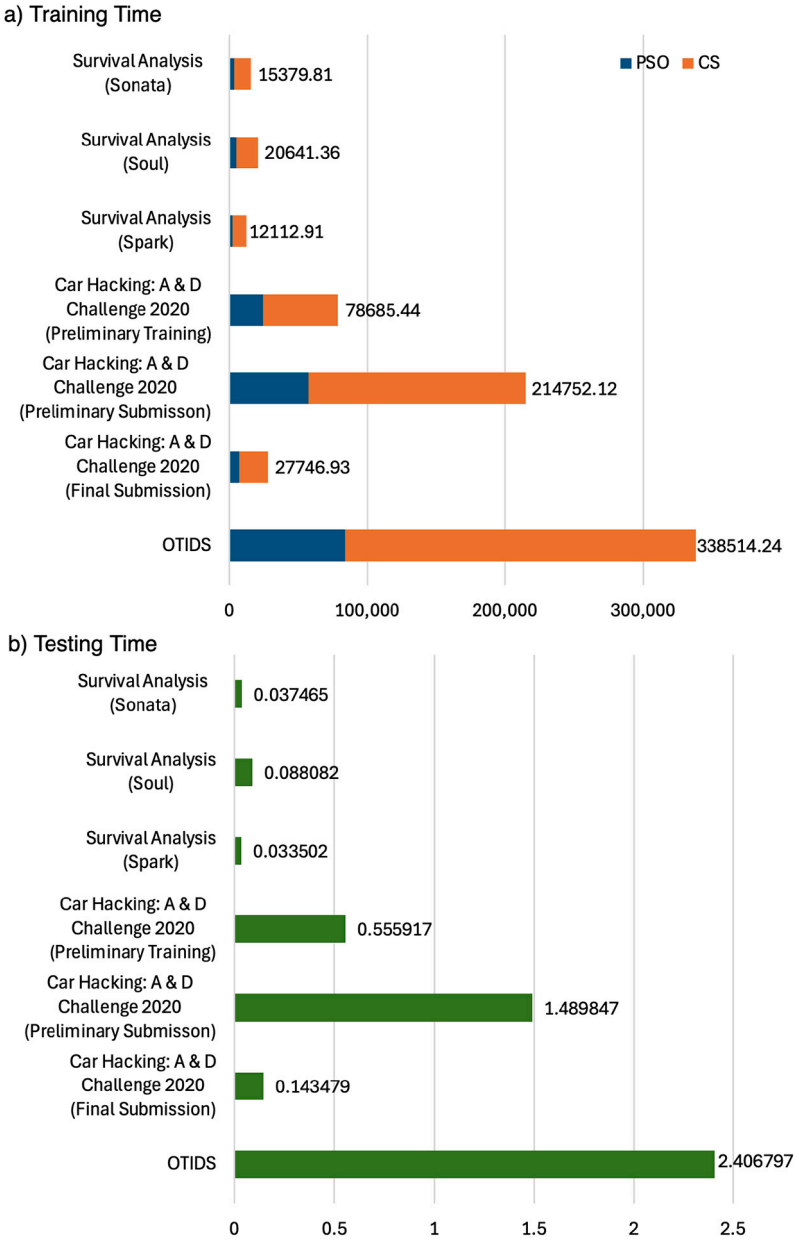


Fig. 15. Graphical Representation of Timing Analysis of MetaCAN (in seconds), including both Training and Testing Durations.

superior performance in detecting five distinct attack types across three real-world datasets (see Table 5, 7 and 8). This shows its effectiveness and robustness in securing the CAN bus.

7.1. Limitations and further development

Despite the strong performance of MetaCAN across the evaluated datasets, further development could strengthen certain aspects. First, the adaptability of MetaCAN to previously unseen or evolving attack patterns remains a challenge, as the current implementation is trained on a fixed set of known attack types. Second, while the system has shown high accuracy and reliability within the tested datasets, these datasets may not fully represent the wide range of possible CAN bus environments and attack scenarios. As such, further validation on more diverse datasets will be useful to thoroughly assess the generalizability and robustness of the proposed model. Third, while MetaCAN achieves efficient testing times and it can perform well in real-time environments post-training, the initial training process is computationally intensive. This is primarily due to the use of metaheuristic optimization algorithms, which increase the training time significantly. Consequently, when retraining is required to incorporate new attack patterns, the training process must be repeated from scratch, potentially limiting the practicality of the model for rapid updates.

7.2. Future research directions

Future research will focus on enhancing the robustness, adaptability, and interpretability of MetaCAN through several key directions, including **(1) Exploring Alternative Ensemble Metaheuristics:** Future research could explore replacing the current PSO or CS optimization strategy with an ensemble metaheuristic approach, such as combining Genetic Algorithm (GA) and Differential Evolution (DE). This could enhance global search capabilities and convergence speed; **(2) Integration of Online Learning:** Incorporating an online learning mechanism such as incremental learning or reinforcement learning could significantly enhance the model's adaptability to evolving threat landscapes. This would allow the MetaCAN to dynamically update itself in real time, improving its robustness and reliability in live deployment scenarios;

(3) Integration of Graphics Processing Units (GPUs): The integration of GPUs will be considered to accelerate model training and inference. Leveraging GPU parallelism can significantly reduce processing time, making the system more efficient. This enhancement would be particularly beneficial when implementing online learning techniques, where rapid updates and low-latency decision-making are crucial for effective deployment in real-world automotive environments; **(4) Adversarial Robustness Evaluation:** To ensure the resilience of the MetaCAN against evasion techniques, future work could involve testing its performance under adversarial conditions using established attack frameworks like Fast Gradient Sign Method (FGSM) and Projected Gradient Descent (PGD). Such evaluations would provide deeper insights into the system's security under more sophisticated threats; and **(5) Incorporation of Explainable AI (XAI):** Providing interpretable insights into decisions of MetaCAN can help enhance user trust and support cybersecurity analysts in identifying attack signatures.

CRediT authorship contribution statement

Kadir Ileri: Writing – original draft, Software, Visualization, Methodology, Funding acquisition, Conceptualization. **Abdur Rakib:** Writing – review & editing, Supervision, Methodology, Conceptualization, Project administration. **Soufiene Djahel:** Writing – review & editing, Formal analysis.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was partially supported by the Scientific and Technological Research Council of Türkiye (TUBITAK) under the 2219 International Postdoctoral Research Fellowship Programme, grant number 1059B192301714.

Data availability

Data will be made available on request.

References

[1] N. Ayres, L. Deka, D. Paluszczyszyn, Container-based electronic control unit virtualisation: a paradigm shift towards a centralised automotive e/e architecture, Electronics 13 (21) (2024) 4283.
[2] H.M. Boland, M.I. Burgett, A.J. Etienne, R.M. Stwalley III, An overview of can-bus development, utilization, and future potential in serial network messaging for off-road mobile equipment, in: Technology in Agriculture, 2021.
[3] S.-F. Lokman, A.T. Othman, M.-H. Abu-Bakar, Intrusion detection system for automotive controller area network (can) bus system: a review, EURASIP J. Wirel. Commun. Netw. 2019 (1) (2019) 1–17.
[4] T. Hoppe, J. Dittman, Sniffing/replay attacks on can buses: a simulated attack on the electric window lift classified using an adapted cert taxonomy, in: Proceedings of the 2nd Workshop on Embedded Systems Security (WESS), 2007, pp. 1–6.
[5] K. Koscher, A. Czeskis, F. Roesner, S. Patel, T. Kohno, S. Checkoway, D. McCoy, B. Kantor, D. Anderson, H. Shacham, et al., Experimental security analysis of a modern automobile, in: 2010 IEEE Symposium on Security and Privacy, IEEE, 2010, pp. 447–462.
[6] T. Hoppe, S. Kiltz, J. Dittmann, Security threats to automotive can networks—practical examples and selected short-term countermeasures, Reliab. Eng. Syst. Saf. 96 (1) (2011) 11–25.
[7] S. Checkoway, D. McCoy, B. Kantor, D. Anderson, H. Shacham, S. Savage, K. Koscher, A. Czeskis, F. Roesner, T. Kohno, Comprehensive experimental analyses of automotive attack surfaces, in: 20th USENIX Security Symposium, in: USENIX Security, vol. 11, 2011.
[8] N. News, Hackers steal bmws in 3 minutes using security loophole, https://www.nbcnews.com/tech/tech-news/hackers-steal-bmws-3-minutes-using-security-loophole-flna868400, 2012. (Accessed 19 December 2024).
[9] S. Woo, H.J. Jo, D.H. Lee, A practical wireless attack on the connected car and security protocol for in-vehicle can, IEEE Trans. Intell. Transp. Syst. 16 (2) (2014) 993–1006.
[10] C. Miller, Remote Exploitation of an Unaltered Passenger Vehicle, Black Hat USA, 2015.
[11] A. Greenberg, Hackers cut a corvette's brakes via a common car gadget, Wired, https://www.wired.com/2015/08/hackers-cut-corvettes-brakes-via-common-car-gadget/, 2015. (Accessed 19 December 2024).
[12] A. Palanca, E. Evenchick, F. Maggi, S. Zanero, A stealth, selective, link-layer denial-of-service attack against automotive networks, in: 14th International DIMVA Conference, Springer, 2017, pp. 185–206.
[13] Y.D.S. Nie, L. Liu, Free-fall: hacking tesla from wireless to can bus, https://www.blackhat.com/docs/us-17/thursday/us-17-Nie-Free-Fall-Hacking-Tesla-From-Wireless-To-CAN-Bus-wp.pdf, 2017. (Accessed 19 December 2024).
[14] Q. Wang, S. Sawhney, Vecure: a practical security framework to protect the can bus of vehicles, in: 2014 International Conference on the Internet of Things (IOT), IEEE, 2014, pp. 13–18.
[15] W. Beniel Dennyson, C. Jothikumar, A review on controller area network and electronic control unit in automotive environment, J. Positive School Psychol. (2022) 269–277.
[16] J. Palomino, E. Cuty, A. Huanachin, Development of a can bus datalogger for recording sensor data from an internal combustion ecu, in: 2021 IEEE International Workshop of Electronics, Control, Measurement, Signals and Their Application to Mechatronics (ECMSM), IEEE, 2021, pp. 1–4.
[17] Z. Ali, Q.I. Ali, An efficient design of a basic autonomous vehicle based on can bus, Int. Trans. Electr. Eng. Comput. Sci. 3 (1) (2024) 41–56.
[18] J. Laufenberg, T. Kropf, O. Bringmann, Can simulation framework-from classic can to can xl, in: 2023 IEEE 26th International Conference on Intelligent Transportation Systems (ITSC), IEEE, 2023, pp. 3343–3348.
[19] E. Choi, H. Song, S. Kang, J.-W. Choi, High-speed, low-latency in-vehicle network based on the bus topology for autonomous vehicles: automotive networking and applications, IEEE Veh. Technol. Mag. 17 (1) (2021) 74–84.
[20] M. Bozdal, A wavelet-based intrusion detection system for controller area network (CAN), Cranfield University, 2021, https://dspace.lib.cranfield.ac.uk/handle/1826/21138.
[21] A. Tomlinson, J. Bryans, S.A. Shaikh, H.K. Kalutarage, Detection of automotive can cyber-attacks by identifying packet timing anomalies in time windows, in: IEEE/IFIP DSN-W, IEEE, 2018, pp. 231–238.
[22] V. Zlomislić, K. Fertalj, V. Sruk, Denial of service attacks, defences and research challenges, Clust. Comput. 20 (2017) 661–671.
[23] F. Fakhfakh, M. Tounsi, M. Mosbah, Cybersecurity attacks on CAN bus based vehicles: a review and open challenges, Library Hi Tech 40 (5) (2022) 1179–1203.
[24] M. Bozdal, M. Samie, S. Aslam, I. Jennions, Evaluation of can bus security challenges, Sensors 20 (8) (2020) 2364.
[25] J. den Hartog, N. Zannone, et al., Security and privacy for innovative automotive applications: a survey, Comput. Commun. 132 (2018) 17–41.
[26] I. Studnia, V. Nicomette, E. Alata, Y. Deswarte, M. Kaâniche, Y. Laarouchi, Survey on security threats and protection mechanisms in embedded automotive networks, in: 2013 43rd Annual IEEE/IFIP Conference on Dependable Systems and Networks Workshop (DSN-W), IEEE, 2013, pp. 1–12.
[27] O. Avatefipour, H. Malik, State-of-the-art survey on in-vehicle network communication (can-bus) security and vulnerabilities, arXiv preprint, arXiv:1802.01725, 2018.
[28] E. Aliwa, O. Rana, C. Perera, P. Burnap, Cyberattacks and countermeasures for in-vehicle networks, ACM Comput. Surv. 54 (1) (2021) 1–37.
[29] B.I. Kwak, M.L. Han, H.K. Kim, Cosine similarity based anomaly detection methodology for the can bus, Expert Syst. Appl. 166 (2021) 114066.
[30] N. Nowdehi, W. Aoudi, M. Almgren, T. Olovsson, Casad: can-aware stealthy-attack detection for in-vehicle networks, arXiv preprint, arXiv:1909.08407, 2019.
[31] H.J.K.A. Masoodi, Evaluating the effectiveness of machine learning-based intrusion detection in multi-cloud environments, Babylonian J. Int. Things 2024 (2024) 94–105.
[32] M.F. Elrawy, A.I. Awad, H.F. Hamed, Intrusion detection systems for iot-based smart environments: a survey, J. Cloud Comput. 7 (1) (2018) 1–20.
[33] K. SCARFONE, Guide to Intrusion Detection and Prevention Systems (IDPS), NIST special publication, 2007.
[34] E.E. Abdallah, A. Aloqaily, H. Fayez, Identifying intrusion attempts on connected and autonomous vehicles: a survey, Proc. Comput. Sci. 220 (2023) 307–314.
[35] I. Studnia, E. Alata, V. Nicomette, M. Kaâniche, Y. Laarouchi, A language-based intrusion detection approach for automotive embedded networks, Int. J. Embed. Syst. 10 (1) (2018) 1–12.
[36] A. Taylor, N. Japkowicz, S. Leblanc, Frequency-based anomaly detection for the automotive can bus, in: 2015 World Congress on Industrial Control Systems Security (WCICSS), IEEE, 2015, pp. 45–49.
[37] W. Choi, K. Joo, H.J. Jo, M.C. Park, D.H. Lee, Voltageids: low-level communication characteristics for automotive intrusion detection system, IEEE Trans. Inf. Forensics Secur. 13 (8) (2018) 2114–2129.
[38] A. Alfardus, D.B. Rawat, Intrusion detection system for can bus in-vehicle network based on machine learning algorithms, in: 2021 IEEE 12th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON), IEEE, 2021, pp. 0944–0949.

[39] H.-C. Lin, P. Wang, K.-M. Chao, W.-H. Lin, J.-H. Chen, Using deep learning networks to identify cyber attacks on intrusion detection for in-vehicle networks, Electronics 11 (14) (2022) 2180.
[40] M.D. Hossain, H. Inoue, H. Ochiai, D. Fall, Y. Kadobayashi, Lstm-based intrusion detection system for in-vehicle can bus communications, IEEE Access 8 (2020) 185489–185502.
[41] A.R. Javed, S. Ur Rehman, M.U. Khan, M. Alazab, T. Reddy, Canintelliids: detecting in-vehicle intrusion attacks on a controller area network using cnn and attention-based gru, IEEE Trans. Netw. Sci. Eng. 8 (2) (2021) 1456–1466.
[42] S.T. Mehedi, A. Anwar, Z. Rahman, K. Ahmed, Deep transfer learning based intrusion detection system for electric vehicular networks, Sensors 21 (14) (2021) 4736.
[43] A. Anjum, P. Agbaje, S. Hounsinou, H. Olufowobi, In-vehicle network anomaly detection using extreme gradient boosting machine, in: 2022 11th Mediterranean Conference on Embedded Computing (MECO), IEEE, 2022, pp. 1–6.
[44] E. Alalwany, I. Mahgoub, Classification of normal and malicious traffic based on an ensemble of machine learning for a vehicle can-network, Sensors 22 (23) (2022) 9195.
[45] M. Driss, I. Almomani, Z. e Huma, J. Ahmad, A federated learning framework for cyberattack detection in vehicular sensor networks, Complex Intell. Syst. 8 (5) (2022) 4221–4235.
[46] D. Caivano, C. Catalano, M. De Vincentiis, A. Lako, A. Pagano, Marea: multi-class random forest for automotive intrusion detection, in: International Conference on Product-Focused Software Process Improvement, Springer, 2023, pp. 23–34.
[47] V. Cobilean, H.S. Mavikumbure, C.S. Wickramasinghe, B.J. Varghese, T. Pennington, M. Manic, Anomaly detection for in-vehicle communication using transformers, in: IECON 2023-49th Annual Conference of the IEEE, Industrial Electronics Society, IEEE, 2023, pp. 1–6.
[48] H.S. Mavikumbure, V. Cobilean, C.S. Wickramasinghe, B.J. Varghese, T. Pennington, M. Manic, Dadae: domain adversarial autoencoder based in-vehicle can anomaly detection, in: IECON 2023-49th Annual Conference of the IEEE, Industrial Electronics Society, IEEE, 2023, pp. 1–7.
[49] S.B. Park, H.J. Jo, D.H. Lee, G-idcs: graph-based intrusion detection and classification system for can protocol, IEEE Access 11 (2023) 39213–39227.
[50] N. Al-Aql, A. Al-Shammari, Hybrid rnn-lstm networks for enhanced intrusion detection in vehicle can systems, J. Electr. Syst. 20 (6s) (2024) 3019–3031.
[51] M.H. Khan, A.R. Javed, Z. Iqbal, M. Asim, A.I. Awad, Divacan: detecting in-vehicle intrusion attacks on a controller area network using ensemble learning, Comput. Secur. 139 (2024) 103712.
[52] J. Xiao, H. Chen, F. Zhong, A novel feature extraction framework using graph node attention network for in-vehicle networks intrusion detection, IEEE Syst. J. (2024).
[53] M.-J. Kang, J.-W. Kang, Intrusion detection system using deep neural network for in-vehicle network security, PLoS ONE 11 (6) (2016) e0155781.
[54] M. Almehdhar, A. Albaseer, M.A. Khan, M. Abdallah, H. Menouar, S. Al-Kuwari, A. Al-Fuqaha, Deep learning in the fast lane: a survey on advanced intrusion detection systems for intelligent vehicle networks, IEEE Open J. Veh. Technol. (2024).
[55] J.-S. Chou, J.P.P. Thedja, Metaheuristic optimization within machine learning-based classification system for early warnings related to geotechnical problems, Autom. Constr. 68 (2016) 65–80.
[56] S. Kiliçarslan, Pso+ gwo: a hybrid particle swarm optimization and grey wolf optimization based algorithm for fine-tuning hyper-parameters of convolutional neural networks for cardiovascular disease detection, J. Ambient Intell. Humaniz. Comput. 14 (1) (2023) 87–97.
[57] S.L. Zubaidi, H. Al-Bugharbee, A.W. Alattabi, H.M. Ridha, K. Hashim, N. Al-Ansari, Z.M. Yaseen, Forecasting urban water demand using different hybrid-based metaheuristic algorithms' inspire for extracting artificial neural network hyperparameters, Sci. Rep. 14 (1) (2024) 24042.
[58] T. Chen, C. Guestrin, Xgboost: a scalable tree boosting system, in: Proceedings of the 22nd Acm Sigkdd International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785–794.
[59] K. Afifah, I.N. Yulita, I. Sarathan, Sentiment analysis on telemedicine app reviews using xgboost classifier, in: 2021 International Conference on Artificial Intelligence and Big Data Analytics, IEEE, 2021, pp. 22–27.
[60] X. Ren, H. Guo, S. Li, S. Wang, J. Li, A novel image classification method with cnn-xgboost model, in: Digital Forensics and Watermarking, 16th International Workshop, IWDW 2017, Magdeburg, Germany, August 23–25, 2017, in: Proceedings, vol. 16, Springer, 2017, pp. 378–390.
[61] A. Asselman, M. Khaldi, S. Aammou, Enhancing the prediction of student performance based on the machine learning xgboost algorithm, Interact. Learn. Environ. 31 (6) (2023) 3360–3379.
[62] A.H. Halim, I. Ismail, S. Das, Performance assessment of the metaheuristic optimization algorithms: an exhaustive review, Artif. Intell. Rev. 54 (3) (2021) 2323–2409.
[63] J. Kennedy, R. Eberhart, Particle swarm optimization, in: Proceedings of ICNN'95-International Conference on Neural Networks, vol. 4, 1995, pp. 1942–1948.
[64] J. Nayak, H. Swapnarekha, B. Naik, G. Dhiman, S. Vimal, 25 years of particle swarm optimization: flourishing voyage of two decades, Arch. Comput. Methods Eng. 30 (3) (2023) 1663–1725.
[65] T.M. Shami, A.A. El-Saleh, M. Alswaitti, Q. Al-Tashi, M.A. Summakieh, S. Mirjalili, Particle swarm optimization: a comprehensive survey, IEEE Access 10 (2022) 10031–10061.
[66] X.-S. Yang, S. Deb, Cuckoo search via Lévy flights, in: 2009 World Congress on Nature & Biologically Inspired Computing (NaBIC), IEEE, 2009, pp. 210–214.
[67] X.-S. Yang, Cuckoo Search and Firefly Algorithm: Theory and Applications, vol. 516, Springer, 2013.
[68] M. Mareli, B. Twala, An adaptive cuckoo search algorithm for optimisation, Appl. Comput. Inform. 14 (2) (2018) 107–115.
[69] A.H. Gandomi, X.-S. Yang, A.H. Alavi, Cuckoo search algorithm: a metaheuristic approach to solve structural optimization problems, Eng. Comput. 29 (2013) 17–35.
[70] X.-S. Yang, S. Deb, Engineering optimisation by cuckoo search, Int. J. Math. Model. Numer. Optim. 1 (4) (2010) 330–343.
[71] S. Mirjalili, S.Z.M. Hashim, H.M. Sardroudi, Training feedforward neural networks using hybrid particle swarm optimization and gravitational search algorithm, Appl. Math. Comput. 218 (22) (2012) 11125–11137.
[72] J.-F. Chen, Q.H. Do, H.-N. Hsieh, Training artificial neural networks by a hybrid pso-cs algorithm, Algorithms 8 (2) (2015) 292–308.
[73] M. Jiang, Y.P. Luo, S.Y. Yang, Stochastic convergence analysis and parameter selection of the standard particle swarm optimization algorithm, Inf. Process. Lett. 102 (1) (2007) 8–16.
[74] F. Wang, X.-S. He, Y. Wang, S.-M. Yang, Markov model and convergence analysis based on cuckoo search algorithm, Comput. Eng. 38 (11) (2012).
[75] H. Lee, S.H. Jeong, H.K. Kim, Otids: a novel intrusion detection system for in-vehicle network by using remote frame, in: 2017 15th Annual Conference on Privacy, Security and Trust (PST), IEEE, 2017, pp. 57–5709.
[76] M.L. Han, B.I. Kwak, H.K. Kim, Anomaly intrusion detection method for vehicular networks based on survival analysis, Veh. Commun. 14 (2018) 52–63.
[77] H. Kang, B.I. Kwak, Y.H. Lee, H. Lee, H. Lee, H.K. Kim, Car Hacking: Attack Defense Challenge 2020 Dataset, IEEE Dataport, 2021.
[78] H. Kang, B.I. Kwak, Y.H. Lee, H. Lee, H. Lee, H.K. Kim, Car hacking and defense competition on in-vehicle network, in: Workshop on Automotive and Autonomous Vehicle Security (AutoSec), vol. 2021, 2021, p. 25.
[79] M. Grandini, E. Bagli, G. Visani, Metrics for multi-class classification: an overview, arXiv preprint, arXiv:2008.05756, 2020.
[80] T.P. Nguyen, H. Nam, D. Kim, Transformer-based attention network for in-vehicle intrusion detection, IEEE Access 11 (2023) 55389–55403.
[81] D. Divya Raj, G. Renjith, S. Aji, A lightweight intrusion detection model for in-vehicular can networks, in: Proceedings of Third International Conference on Sustainable Expert Systems: ICSES 2022, Springer, 2023, pp. 665–678.
[82] G.A. Al-Absi, Y. Fang, A.A. Qaseem, Stc-graphformer: graph spatial-temporal correlation transformer for in-vehicle network intrusion detection system, Veh. Commun. 51 (2025) 100865.
[83] N. Khatri, S. Lee, S.Y. Nam, Transfer learning-based intrusion detection system for a controller area network, IEEE Access 11 (2023) 120963–120982.
[84] K. Okokpujie, D. Kalimumbalo, J.A. Badejo, E. Adetiba, Congestion intrusion detection-based method for controller area network bus: a case for kia soul vehicle, in: Mathematical Modelling of Engineering Problems, 2022, https://doi.org/10.18280/mmep.090518.
[85] S. Bajpai, K. Sharma, B.K. Chaurasia, Intrusion detection for Internet of vehicles using machine learning, in: 2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT), IEEE, 2023, pp. 1–6.
[86] W. Lalouani, I. Omorogbe, C. Dube, M. Baza, A. Rasheed, K.-K.R. Choo, A robust physical layer ids for intra-vehicle communication security, in: 2024 International Conference on Smart Applications, Communications and Networking (SmartNets), IEEE, 2024, pp. 1–6.
[87] Y. He, J. Gao, M. Fan, D. Han, Y. Lu, Y. Qiao, A&d graph-based graph neural network intrusion detection for in-vehicle controller area network, in: 2024 IEEE/CIC International Conference on Communications in China (ICCC), IEEE, 2024, pp. 1281–1286.